

中南大學

数据结构与算法实验报告

专业班级	电信 2303
学号姓名	8207230322-樊博文
指导教师	陈科文
完成日期	2026. 1. 8

中南大学 · 自动化学院

目录

实验一：线性表操作及其应用算法设计	4
任务 1（必做）	4
1. 实验项目名称	4
2. 实验目的	4
3. 实验任务描述	4
4. 任务分析及算法设计思路	5
5. 程序结构及代码解析	7
6. 实验测试数据和实验结果分析	13
7. 实验总结与体会	13
任务 2（选做）	15
1. 实验项目名称	15
2. 实验目的	15
3. 实验任务描述	15
4. 任务分析及算法设计思路	15
5. 程序结构及代码解析	18
6. 实验测试数据和实验结果分析	22
7. 实验总结与体会	24
实验二：二叉树的有关算法设计	26
任务 1（必做）	26
1. 实验项目名称	26
2. 实验目的	26
3. 实验任务描述	26
4. 任务分析及算法设计思路	27
5. 程序结构及代码解析	29
6. 实验测试数据和实验结果分析	33
7. 实验总结与体会	34
任务 3（选做）	36
1. 实验项目名称	36
2. 实验目的	36
3. 实验任务描述	36
4. 任务分析及算法设计思路	37
5. 程序结构及代码解析	38
6. 实验测试数据和实验结果分析	44
7. 实验总结与体会	45
实验三：图的有关算法设计	46
任务 1（必做）	46
1. 实验项目名称	46
2. 实验目的	46
3. 实验任务描述	46

4. 任务分析及算法设计思路	47
5. 程序结构及代码解析	48
6. 实验测试数据和实验结果分析	55
7. 实验总结与体会	56
任务 2 (选做)	57
1. 实验项目名称	57
2. 实验目的	57
3. 实验任务描述	57
4. 任务分析及算法设计思路	57
5. 程序结构及代码解析	58
6. 实验测试数据和实验结果分析	62
7. 实验总结与体会	63
实验四：查找与排序算法设计及应用	65
(一) 查找算法设计及其应用	65
1. 实验项目名称	65
2. 实验目的	65
3. 实验任务描述	65
4. 任务分析及算法设计思路	65
5. 程序结构及代码解析	67
6. 实验测试数据和实验结果分析	70
7. 实验总结与体会	72
(二) 排序算法设计及其应用	73
1. 实验项目名称	73
2. 实验目的	73
3. 实验任务描述	73
4. 任务分析及算法设计思路	73
5. 程序结构及代码解析	75
6. 实验测试数据和实验结果分析	81
7. 实验总结与体会	84

实验一：线性表操作及其应用算法设计

要求：有 3 个任务，一个必做实验，两个选做实验（任选其一）。

任务 1（必做）

实现循环单链表的各种基本运算的算法，实验内容见主教材（第六版）第 74 页的实验题 4。

1.实验项目名称

实现循环单链表的各种基本运算的算法。

2.实验目的

领会循环单链表的存储结构和掌握循环单链表中各种基本运算算法的设计。

3.实验任务描述

编写一个程序 `clinklist.cpp`，实现循环单链表的各种基本运算和整体链表算法（假设循环单链表的元素类型 `ElemType` 为 `int`），并在此基础上设计一个程序 `exp2-4.cpp` 完成以下功能：

1. 初始化循环链表 h。
2. 依次采用尾插法插入 a、b、c、d、e 元素。
3. 输出循环链表 h。
4. 输出循环链表 h 的长度。
5. 判断循环链表 h 是否为空。
6. 输出循环链表 h 的第 3 个元素。
7. 输出元素 a 的位置。
8. 在第 4 个元素的位置上插入 f 元素。
9. 输出循环链表 h。
10. 删除循环链表 h 的第 3 个元素。
11. 输出循环链表 h。

12. 释放循环单链表 h。

4.任务分析及算法设计思路

4.1 任务分析

本实验要求完成带头结点的循环单链表的基本运算，并在测试程序中按给定步骤依次完成：初始化、尾插建表、遍历输出、求长度、判空、按位取值、按值定位、按位插入、按位删除以及销毁链表等操作。

循环单链表的核心特征是：尾结点的 next 指向头结点，链表首尾相接形成闭环。由于不存在 NULL 结尾，遍历和停止条件必须以“是否回到头结点”来判断，这与普通单链表（以 NULL 为结束标志）不同。

为降低插入/删除的边界处理难度，本实验采用带头结点结构：

- 头结点不存放有效数据，仅作为统一入口与遍历终止标志；
- 空表时满足 $L \rightarrow next == L$ ；
- 非空表从 $L \rightarrow next$ 出发沿 next 指针移动最终回到 L。

4.2 数据结构设计（存储结构）

结点结构：

- data：存放元素值（int）
- next：指向下一结点

循环链表应始终满足的不变式：

头结点 L 作为循环标记存在；

空表： $L \rightarrow next == L$ ；

非空表：从 $L \rightarrow next$ 出发遍历，最终回到 L；

任意插入、删除之后仍保持闭环结构，不出现断链或非法指针。

4.3 基本操作设计思想

初始化（InitList）动态申请头结点空间，使 $L \rightarrow next = L$ ，建立空循环链表。

尾插建表/追加（CreateListR）使用尾指针思想：先定位到当前尾结点，再逐个创建新结点并链接到尾部，保证新结点的 next 指回头结点 L，从而维持循环结构。

该方法既适用于从空表创建，也适用于在已有链表末尾继续追加元素。

遍历输出（DispList）从首元结点 $L \rightarrow next$ 开始遍历，循环条件为 $p \neq L$ 。一旦

指针回到头结点，说明完成一圈遍历并结束。

求长度 (ListLength) /判空 (ListEmpty) 判空: $L == \text{NULL}$ 或 $L \rightarrow \text{next} == L$

求长: 遍历一圈计数, 直到回到头结点

按位取值 (GetElem) 采用 1-based 位序, 从第一个有效结点开始计数, 走到第 i 个结点取其 data; 若中途回到头结点则说明越界。

按值定位 (LocateElem) 从首元结点开始遍历, 遇到 $\text{data} == e$ 返回当前位置; 遍历回到头结点仍未找到则返回 0。

按位插入 (ListInsert) 插入的关键在于先找到第 $i-1$ 个结点 (前驱) p , 然后将新结点 s 插入在 p 与 $p \rightarrow \text{next}$ 之间: $s \rightarrow \text{next} = p \rightarrow \text{next}$; $p \rightarrow \text{next} = s$; 若 i 超出合法范围则插入失败。

按位删除 (ListDelete) 与插入类似, 先找到第 $i-1$ 个结点 (前驱) p , 令 $q = p \rightarrow \text{next}$ 为待删结点, 执行: $p \rightarrow \text{next} = q \rightarrow \text{next}$; $\text{free}(q)$; 并通过引用参数返回被删元素值。

销毁链表 (DestroyList) 从首元结点开始依次释放所有有效结点, 最后释放头结点并将 L 置空, 避免野指针。

4.4 关键操作

- 遍历模板

```
if L == NULL or L.next == L:
```

```
    空表处理
```

```
p = L.next
```

```
while p != L:
```

```
    访问 p.data
```

```
    p = p.next
```

- 尾插建表

若 L 为空: 初始化头结点

```
r = L
```

```
while r.next != L:
```

```
    r = r.next          // 找到尾结点
```

```
for i = 0..n-1:
```

```
    创建新结点 s
```

```
    s.data = a[i]
```

```
    s.next = L
```

```

    r.next = s
    r = s
• 按位删除
p = L, j = 0
while p.next != L and j < i-1:
    p = p.next; j++
if p.next == L or j != i-1: 失败
q = p.next
e = q.data
p.next = q.next
释放 q
成功

```

5.程序结构及代码解析

5.1 文件结构与功能划分

clinklist.cpp

定义循环单链表结点结构与基本运算函数，包括：初始化、尾插建表、输出、求长、判空、按位取值、按值定位、插入、删除、销毁等。

```

#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

typedef int ElemType;

typedef struct LNode {
    ElemType data;
    struct LNode *next;
} LinkNode;

// 1. 初始化循环链表
void InitList(LinkNode *&L) {
    L = (LinkNode *)malloc(sizeof(LinkNode));
    if (!L) return;
    L->next = L; // 指向自己形成循环
}

// 2. 尾插法创建/追加循环链表（更稳：可在已有链表末尾继续追加）
void CreateListR(LinkNode *&L, const ElemType a[], int n) {
    if (L == NULL) InitList(L);
    if (L == NULL) return;

```

```

    LinkNode *r = L;
    // 找到当前尾结点
    while (r->next != L) r = r->next;

    for (int i = 0; i < n; i++) {
        LinkNode *s = (LinkNode *)malloc(sizeof(LinkNode));
        if (!s) return;
        s->data = a[i];
        s->next = L;
        r->next = s;
        r = s;
    }
}

// 3. 输出循环链表（按字符显示: a b c...）
void DispList(LinkNode *L) {
    if (L == NULL || L->next == L) {
        printf("空链表\n");
        return;
    }
    LinkNode *p = L->next;
    while (p != L) {
        printf("%c ", (char)p->data);
        p = p->next;
    }
    printf("\n");
}

// 4. 获取循环链表长度
int ListLength(LinkNode *L) {
    if (L == NULL) return 0;
    int len = 0;
    LinkNode *p = L->next;
    while (p != L) {
        len++;
        p = p->next;
    }
    return len;
}

// 5. 判断循环链表是否为空
bool ListEmpty(LinkNode *L) {
    return (L == NULL) || (L->next == L);
}

```



```

}

// 6. 获取第 i 个元素 (1-based)
bool GetElem(LinkNode *L, int i, ElemType &e) {
    if (L == NULL || i < 1) return false;
    LinkNode *p = L->next;
    int j = 1;
    while (p != L && j < i) {
        p = p->next;
        j++;
    }
    if (p == L) return false;
    e = p->data;
    return true;
}

// 7. 查找元素 e 的位置 (1-based, 找不到返回 0)
int LocateElem(LinkNode *L, ElemType e) {
    if (L == NULL) return 0;
    LinkNode *p = L->next;
    int i = 1;
    while (p != L) {
        if (p->data == e) return i;
        p = p->next;
        i++;
    }
    return 0;
}

// 8. 在第 i 个位置插入元素 e (1-based, 可在尾部插入 i=len+1)
bool ListInsert(LinkNode *&L, int i, ElemType e) {
    if (L == NULL || i < 1) return false;

    LinkNode *p = L;
    int j = 0; // 头结点视作第 0 个

    // 找到第 i-1 个结点
    while (p->next != L && j < i - 1) {
        p = p->next;
        j++;
    }
    if (j != i - 1) return false; // i 超界

    LinkNode *s = (LinkNode *)malloc(sizeof(LinkNode));

```

```

    if (!s) return false;

    s->data = e;
    s->next = p->next;
    p->next = s;
    return true;
}

// 9. 删除第 i 个元素 (1-based)
bool ListDelete(LinkNode *&L, int i, ElemType &e) {
    if (L == NULL || i < 1) return false;

    LinkNode *p = L;
    int j = 0;

    // 找到第 i-1 个结点
    while (p->next != L && j < i - 1) {
        p = p->next;
        j++;
    }
    if (p->next == L || j != i - 1) return false; // i 超界/不存在

    LinkNode *q = p->next;
    e = q->data;
    p->next = q->next;
    free(q);
    return true;
}

// 10. 释放循环链表
void DestroyList(LinkNode *&L) {
    if (L == NULL) return;

    LinkNode *p = L->next;
    while (p != L) {
        LinkNode *q = p;
        p = p->next;
        free(q);
    }
    free(L);
    L = NULL;
}

```

exp2-4. cpp

测试驱动程序，按实验要求 1~12 步依次调用 clinklist.cpp 中的函数并打印结果。（对应文件 test1-1.cpp）

```
#include <stdio.h>
#include "clinklist.cpp"

int main() {
    LinkNode *h = NULL;
    ElemType e = 0;

    // 1. 初始化循环链表 h
    printf(" (1) 初始化循环链表 h\n");
    InitList(h);

    // 2. 依次采用尾插法插入 a、b、c、d、e 元素
    printf(" (2) 依次采用尾插法插入 a、b、c、d、e 元素\n");
    ElemType arr[] = {'a', 'b', 'c', 'd', 'e'};
    CreateListR(h, arr, 5);

    // 3. 输出循环链表 h
    printf(" (3) 输出循环链表 h: ");
    DispList(h);

    // 4. 输出循环链表 h 的长度
    printf(" (4) 循环链表 h 长度: %d\n", ListLength(h));

    // 5. 判断循环链表 h 是否为空
    printf(" (5) 循环链表 h 是否为空: %s\n", ListEmpty(h) ? "是" : "否");

    // 6. 输出循环链表 h 的第 3 个元素
    if (GetElem(h, 3, e)) {
        printf(" (6) 循环链表 h 的第 3 个元素: %c\n", (char)e);
    } else {
        printf(" (6) 获取第 3 个元素失败\n");
    }

    // 7. 输出元素 a 的位置
    int pos = LocateElem(h, 'a');
    printf(" (7) 元素 a 的位置: %d\n", pos);

    // 8. 在第 4 个元素的位置上插入 f 元素
    if (ListInsert(h, 4, 'f')) {
        printf(" (8) 在第 4 个元素的位置上插入 f 元素\n");
    } else {
```

```

        printf(" (8) 插入失败\n");
    }

    // 9. 输出循环链表 h
    printf(" (9) 输出循环链表 h: ");
    DispList(h);

    // 10. 删除循环链表 h 的第 3 个元素
    if (ListDelete(h, 3, e)) {
        printf(" (10) 删除循环链表 h 的第 3 个元素: %c\n", (char)e);
    } else {
        printf(" (10) 删除第 3 个元素失败\n");
    }

    // 11. 输出循环链表 h
    printf(" (11) 输出循环链表 h: ");
    DispList(h);

    // 12. 释放循环单链表 h
    DestroyList(h);
    printf(" (12) 释放循环链表 h\n");

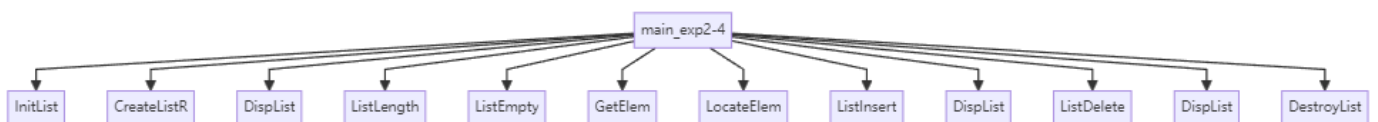
    return 0;
}

```

5.2 函数调用关系图

main()

- |— InitList(h)
- |— CreateListR(h, a, 5)
- |— DispList(h)
- |— ListLength(h)
- |— ListEmpty(h)
- |— GetElem(h, 3, e)
- |— LocateElem(h, 'a')
- |— ListInsert(h, 4, 'f')
- |— DispList(h)
- |— ListDelete(h, 3, e)
- |— DispList(h)
- |— DestroyList(h)



6. 实验测试数据和实验结果分析

6.1 测试数据

程序内置测试序列：a, b, c, d, e，采用尾插法建立循环单链表；随后在第 4 个位置插入 f，并删除第 3 个元素。

6.2 结果分析

初始输出：a b c d e，长度应为 5，判空为“否”。

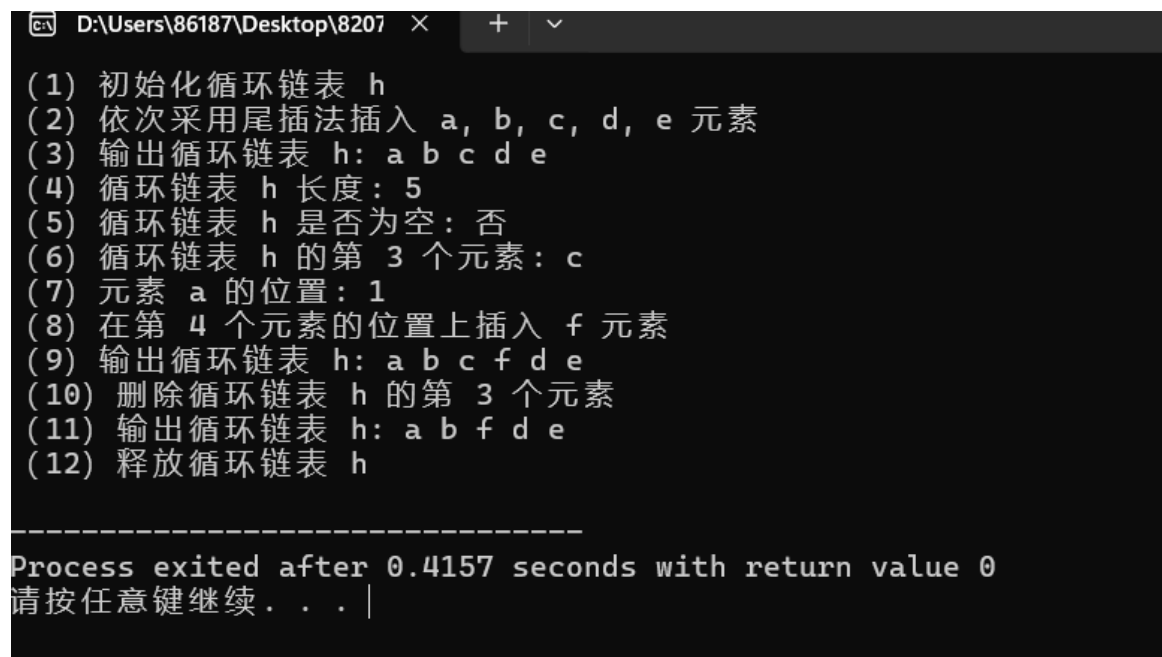
第 3 个元素应为 c，元素 a 的位置应为 1。

插入 f 到第 4 位后：a b c f d e。

删除第 3 个元素 (c) 后：a b f d e。

程序输出与预期一致，说明循环单链表的基本运算实现正确。

6.3 程序运行结果截图



```
D:\Users\86187\Desktop\8207  X + v
(1) 初始化循环链表 h
(2) 依次采用尾插法插入 a, b, c, d, e 元素
(3) 输出循环链表 h: a b c d e
(4) 循环链表 h 长度: 5
(5) 循环链表 h 是否为空: 否
(6) 循环链表 h 的第 3 个元素: c
(7) 元素 a 的位置: 1
(8) 在第 4 个元素的位置上插入 f 元素
(9) 输出循环链表 h: a b c f d e
(10) 删除循环链表 h 的第 3 个元素
(11) 输出循环链表 h: a b f d e
(12) 释放循环链表 h

-----
Process exited after 0.4157 seconds with return value 0
请按任意键继续. . . |
```

7. 实验总结与体会

实验收获

本次实验让我对循环单链表的结构与操作方式形成了清晰的整体认识。循环单链表与普通单链表最关键的差异在遍历终止条件：普通单链表以 NULL 作为结束标志，而循环单链表需要以指针回到头结点作为结束标志。看起来只是判断条件不同，

但它直接影响遍历、查找、插入、删除等所有操作的编写方式与正确性验证。实验中我也更直观地体会到带头结点的优势，头结点既统一了链表的入口，又为遍历提供了明确的停止位置，使插入和删除都能围绕前驱结点展开，减少对首元结点的特殊处理，代码更简洁，逻辑更统一。通过实现插入与删除操作，我对链表的本质操作也理解得更扎实：核心不在于找到目标结点本身，而在于准确定位其前驱结点，通过修改 `next` 指针完成指针重连，并在操作后维持尾结点指向头结点的闭环结构，从而保证链表结构始终正确。

遇到的问题及解决办法

实验中遇到的主要问题集中在遍历逻辑。循环单链表不以 `NULL` 结尾，若遍历仍使用普通单链表的写法，或建表、插入后未保持尾结点指向头结点，会导致遍历无法结束、输出停滞。解决时我统一了遍历规范：从首元结点开始访问，循环条件固定为 `p != L`，并在建表与插入等结构性操作结束后保证链表闭环不被破坏，从根本上消除死循环问题。另一个容易出错的地方是插入与删除的位序边界处理。位序不合法时若没有及时判断，会出现把头结点当作有效结点访问或访问越界结点的情况，造成链接错误。为此我在插入与删除中都先定位第 `i-1` 个结点作为前驱，通过计数确认前驱位置合法后再进行指针重连，保证操作安全且逻辑一致。

任务 2（选做）

用栈求解迷宫问题的所有路径及最短路径,实验内容见主教材(第六版)第 118 页:实验题 5。

1.实验项目名称

用栈求解迷宫问题的所有路径及最短路径

2.实验目的

掌握在求解迷宫问题中的应用。

3.实验任务描述

编写一个程序 `exp3-5.cpp`，改进 3.1.4 节的求解迷宫问题程序，要求输出如图 3.28 所示的迷宫的所有路径，并求出第一条最短路径及其长度。

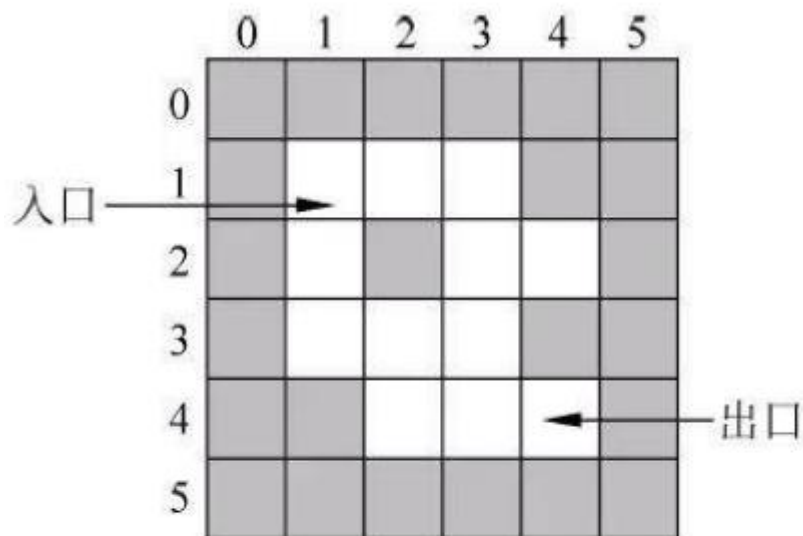


图 3.28 迷宫示意图

4.任务分析及算法设计思路

4.1 任务分析

迷宫可抽象成一个网格图：

- $mg[i][j] == 1$ 表示墙，不能通过

- `mg[i][j] == 0` 表示路，可以通过
- 程序搜索从入口到出口的所有通路，每条通路是一串连续的格子坐标。

由于要求输出所有路径，不能像只求一条路径那样找到出口就结束，而应继续回溯并尝试其他分支。此外，还要求求出“第一条最短路径”：即在遍历所有路径过程中，用“当前路径长度”不断更新最短记录，并保存对应的路径序列。

本实验采用非递归 DFS + 栈回溯：

- 栈 `St` 记录当前探索路径；
- 每个结点包含 `(i, j)` 坐标与 `di`（该结点已经尝试到哪个方向）；
- 当到达出口：输出当前栈形成的路径，并比较更新最短路径；
- 当走入死路或四个方向都试完：出栈回退，并将该点恢复为可走，以便其它路径能再次经过它（保证能枚举所有路径）。

4.2 数据结构设计

```
struct Node {
    int i, j;    // 当前格子坐标
    int di;      // 当前格子已尝试的方向序号（-1 表示尚未尝试）
};
```

- `St[MaxSize]`：栈，保存当前正在搜索的路径
- `Path[MaxSize]`：保存当前找到的最短路径
- `top`：栈顶指针
- `minlen`：当前最短路径长度
- `countPath`：路径编号，用于输出“第几条路径”

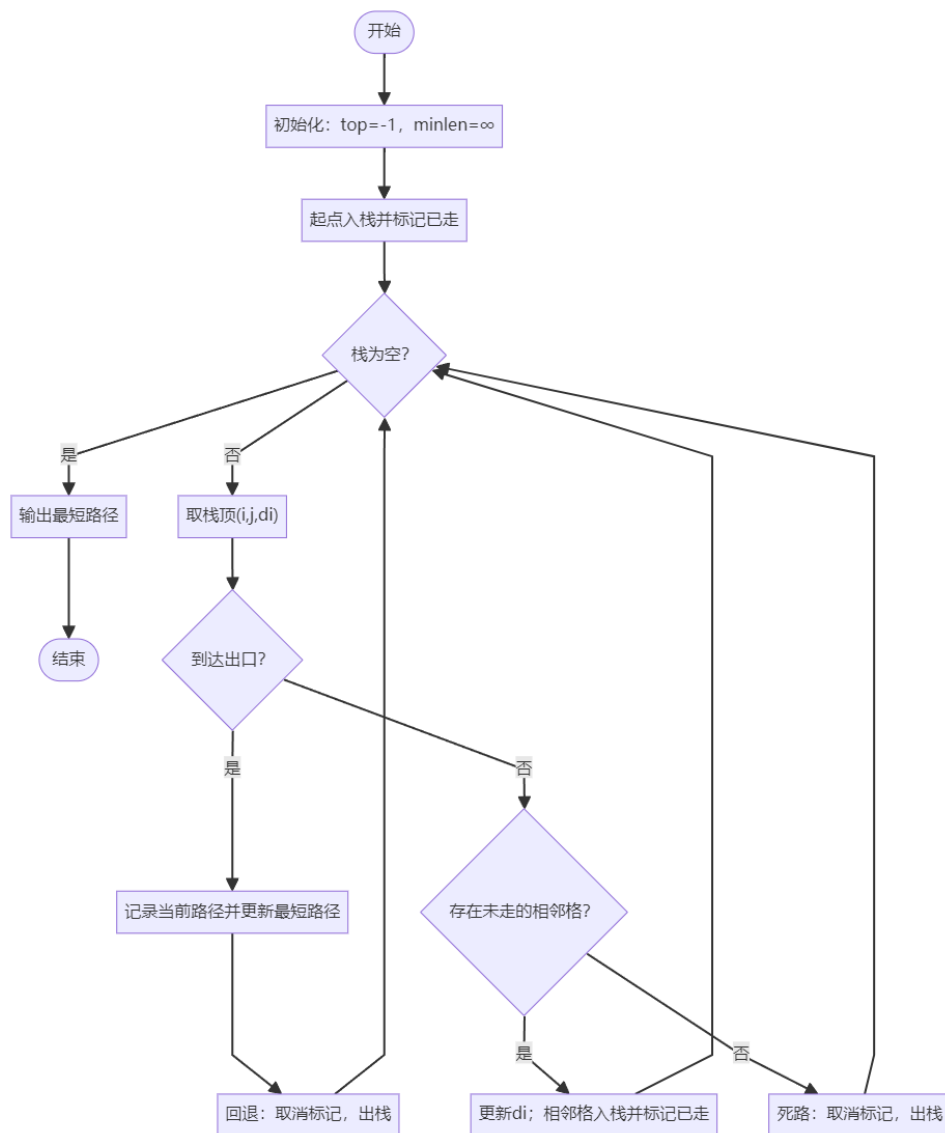
方向数组（上、右、下、左）：

```
int dii[4] = {-1, 0, 1, 0};
```

```
int djj[4] = {0, 1, 0, -1};
```

迷宫外围加一圈 1 作为边框墙，用于避免越界判断，使得访问邻接格子时更安全。

4.3 算法基本流程



非递归 DFS + 回溯输出所有路径:

初始化栈，把入口 (x_i, y_i) 入栈，入口标记为已走(-1)

while 栈不空:

 取栈顶 (i, j, di)

 if (i, j) 是出口:

 输出当前栈路径

 若当前路径更短，则更新最短路径

 恢复出口为可走

 出栈回退

 continue

 从 $di+1$ 开始依次尝试 4 个方向:

 若找到一个邻接格是可走(0):

 更新栈顶 di

 邻接格入栈， $di=-1$

 标记邻接格为已走(-1)

break

若 4 个方向都不能走：
恢复当前格为可走(0)
出栈回退

最终输出记录到的最短路径及长度

关键点：

- “已走标记”用于避免当前路径中重复走同一个格子导致死循环；
- 回溯时恢复为 0，确保其它路径仍可经过该格子，从而能枚举所有路径；
- 用 di 记录“方向尝试进度”，保证回溯后能接着试剩余方向，而不是从头再试。

5.程序结构及代码解析

5.1 程序结构

本实验只有一个源文件 exp3-5.cpp，主要由以下模块组成：

- 迷宫数据与常量定义：mg、M、N、MaxSize
- 结点结构：Node
- 栈与全局变量：St、Path、top、minlen、countPath
- 输出模块：

printCoords：输出坐标序列

printMazeWithPath：将路径在迷宫中用 * 标记输出

printBoth：坐标 + 迷宫图同时输出

- 路径处理模块：

dispath：输出一条完整路径，并更新最短路径

dispathmin：输出最短路径与长度

- 核心算法：

mgpath(xi, yi, xe, ye)：非递归 DFS 枚举所有路径

- 主函数：

main()：调用 mgpath(1, 1, M, N) 并输出结果

```

#include <iostream>
using namespace std;

#define M 4
#define N 4
#define MaxSize 100

// 1=墙, 0=路; 外围加一圈 1 作边框
int mg[M + 2][N + 2] = {
    {1, 1, 1, 1, 1, 1},
    {1, 0, 0, 0, 1, 1},
    {1, 0, 1, 0, 0, 1},
    {1, 0, 0, 0, 1, 1},
    {1, 1, 0, 0, 0, 1},
    {1, 1, 1, 1, 1, 1}
};

struct Node {
    int i, j;
    int di; // 方向索引
};

Node St[MaxSize], Path[MaxSize];
int top = -1;
int countPath = 1;
int minlen = MaxSize;

// 方向: 上、右、下、左
int dii[4] = {-1, 0, 1, 0};
int djj[4] = {0, 1, 0, -1};

// 打印坐标序列
void printCoords(const Node st[], int len) {
    for (int k = 0; k < len; k++)
        cout << "(" << st[k].i << ", " << st[k].j << ") ";
    cout << "\n";
}

// 打印迷宫 (#=墙, 空格=路, *=路径)
void printMazeWithPath(const Node st[], int len) {
    int show[M + 2][N + 2];
    for (int i = 0; i < M + 2; i++)
        for (int j = 0; j < N + 2; j++)
            show[i][j] = mg[i][j];

```

```

    for (int k = 0; k < len; k++) {
        show[st[k].i][st[k].j] = 2;
    }

    for (int i = 1; i <= M; i++) {
        for (int j = 1; j <= N; j++) {
            if (show[i][j] == 1) cout << "# ";
            else if (show[i][j] == 2) cout << "* ";
            else cout << " ";
        }
        cout << "\n";
    }
}

// 同时输出: 坐标 + 迷宫图
void printBoth(const Node st[], int len) {
    cout << "坐标序列: ";
    printCoords(st, len);
    cout << "迷宫标记 (#墙, *路径): \n";
    printMazeWithPath(st, len);
}

void dispapath() {
    int len = top + 1;
    cout << "第" << countPath++ << "条路径: \n";
    printBoth(St, len);
    cout << "\n";

    if (len < minlen) {
        for (int k = 0; k < len; k++) Path[k] = St[k];
        minlen = len;
    }
}

// 输出第一条最短路径
void dispminpath() {
    cout << "最短路径如下: \n";
    cout << "长度: " << minlen << "\n";
    printBoth(Path, minlen);
}

// 非递归 DFS + 回溯
void mgpath(int xi, int yi, int xe, int ye) {

```

```

int i, j, i1, j1, di;
bool find;

top++;
St[top].i = xi; St[top].j = yi; St[top].di = -1;
mg[xi][yi] = -1; // 已走

while (top > -1) {
    i = St[top].i; j = St[top].j;
    di = St[top].di;

    // 到达出口
    if (i == xe && j == ye) {
        dispapath();
        mg[i][j] = 0;    // 恢复出口
        top--;          // 回退
        continue;
    }

    find = false;
    while (di < 3 && !find) {
        di++;
        i1 = i + dii[di];
        j1 = j + djj[di];
        if (mg[i1][j1] == 0) find = true;
    }

    if (find) {
        St[top].di = di;    // 更新当前格已尝试方向
        top++;
        St[top].i = i1; St[top].j = j1; St[top].di = -1;
        mg[i1][j1] = -1;    // 进栈并标记走过
    } else {
        mg[i][j] = 0;    // 死路，恢复为可走
        top--;
    }
}

dispminpath();
}

int main() {
    cout << "迷宫所有路径如下: \n\n";
    mgpath(1, 1, M, N);
}

```

```
    return 0;
}
```

5.2 关键函数说明

(1) 路径输出与最短路径更新: dispapath()

当前路径长度 $len = top + 1$

输出“第 countPath 条路径”，并打印坐标与迷宫标记图

若 $len < minlen$: 说明找到更短路径，将当前栈内容复制到 Path，并更新 minlen

(2) 核心搜索函数: mgpath()

入口入栈，设置 $di=-1$ 表示未尝试方向

进入循环：只要栈非空就不断探索

到出口就输出路径并回退继续找其它路径

否则尝试从当前结点的下一个方向开始寻找可走邻格

找到可走邻格就入栈并标记为已走

4 方向都不可走则说明死路，恢复并出栈回溯

(3) 迷宫打印函数: printMazeWithPath()

复制迷宫到 show 数组

将路径上的坐标位置置为 2

输出时：

1 显示 #（墙）

2 显示 *（路径）

0 显示空格（可走但不在路径上）

6.实验测试数据和实验结果分析

6.1 测试数据

迷宫大小：M=4，N=4，外围加边框墙。

迷宫数组 mg 如程序中所示（1=墙，0=路），入口为 (1, 1)，出口为 (4, 4)。

方向顺序固定为：上 → 右 → 下 → 左。

因此输出路径的顺序与 DFS 搜索顺序一致。

6.2 实验结果分析

程序运行后输出：

1. 迷宫从入口到出口的所有路径，每条路径包含：
 - 坐标序列（按走过顺序输出）
 - 用 * 标记的迷宫路径图
2. 在输出全部路径的过程中，程序会比较每条路径的长度，并更新最短记录。
3. 最终输出：
 - “最短路径如下”
 - 最短路径长度 minlen
 - 最短路径坐标序列与迷宫图

结果正确性的依据：

- 每条路径都是由相邻格移动得到，不穿墙、不越界；
- 当前路径用 -1 标记避免重复走格子，保证路径合法且不会死循环；
- 回溯时恢复为 0，确保能遍历全部可能分支，从而输出所有路径；
- minlen 在所有路径中取最小值，因此得到的最短路径长度正确，且保存的是首次出现的最短路径（即“第一条最短路径”）。

6.3 测试结果

```
D:\Users\86187\Desktop\8207 × + ▾

迷宫所有路径如下：

第1条路径：
坐标序列：(1,1) (1,2) (1,3) (2,3) (3,3) (4,3) (4,4)
迷宫标记（#墙，*路径）：
* * * #
  # *
    * #
#   * *

第2条路径：
坐标序列：(1,1) (1,2) (1,3) (2,3) (3,3) (3,2) (4,2) (4,3) (4,4)
迷宫标记（#墙，*路径）：
* * * #
  # *
    * * #
# * * *

第3条路径：
坐标序列：(1,1) (2,1) (3,1) (3,2) (3,3) (4,3) (4,4)
迷宫标记（#墙，*路径）：
*   #
* #
* * * #
#   * *

第4条路径：
坐标序列：(1,1) (2,1) (3,1) (3,2) (4,2) (4,3) (4,4)
迷宫标记（#墙，*路径）：
*   #
* #
* *   #
# * * *

最短路径如下：
长度：7
坐标序列：(1,1) (1,2) (1,3) (2,3) (3,3) (4,3) (4,4)
迷宫标记（#墙，*路径）：
* * * #
  # *
    * #
#   * *

-----
Process exited after 0.6915 seconds with return value 0
请按任意键继续. . . |
```

7.实验总结与体会

实验收获

通过本实验，我更清晰地理解了栈在迷宫求解中的作用：栈不仅用于保存已经走过的路径，还承担着回溯时恢复现场的功能。相比递归 DFS，非递归实现需要手

动维护当前结点尝试方向进度，因此结构更清晰，也更有助于理解回溯过程本质：不断尝试分支，失败则退回上一状态继续尝试其它方向。

此外，在输出所有路径的基础上统计最短路径，使我体会到搜索 + 记录最优解的经典模式：搜索负责枚举解空间，记录模块负责在枚举过程中维护最优结果。

遇到的问题及解决办法

容易陷入死循环：循环遍历时若不标记已走格子，可能在空地间反复绕圈。

解决：进入格子时将 `mg[i][j]` 标记为 `-1`，避免重复进入同一格。

无法输出所有路径，如果找到出口后直接结束或不恢复标记，会导致后续路径无法枚举。到达出口后输出路径，但不终止程序；同时回溯前恢复出口为可走，并继续搜索其它分支。

回溯后方向重复尝试：如果不保存方向进度，回退到某结点时会从第一个方向重新试，导致重复搜索甚至错误。解决：在结点结构中加入 `di`，每次找到可走方向后更新 `St[top].di`，回溯回来能接着试下一个方向。

实验二：二叉树的有关算法设计

要求：有 3 个任务，一个必做实验，两个选做实验（任选其一）。

任务 1（必做）

实现二叉树的各种基本运算的算法，实验内容见主教材（第六版）第 243 页的实验题 1。

1.实验项目名称

实现二叉树的各种基本运算的算法

2.实验目的

领会二叉链存储结构和掌握二叉树中基本运算算法的设计。

3.实验任务描述

编写一个程序 `btree.cpp`，实现二叉树的基本运算，并在此基础上设计一个程序 `exp7-1.cpp` 完成以下功能。

（1）由图 7.34 所示的二叉树创建对应的二叉链存储结构 `b`，该二叉树的括号表示串为

“`A(B(D, E(H(J, K(L, M(, N))))), C(F, G(, I))`”

（2）输出二叉树 `b`。

（3）输出 'H' 结点的左、右孩子结点信息。

（4）输出二叉树 `b` 的高度。

（5）释放二叉树 `b`。

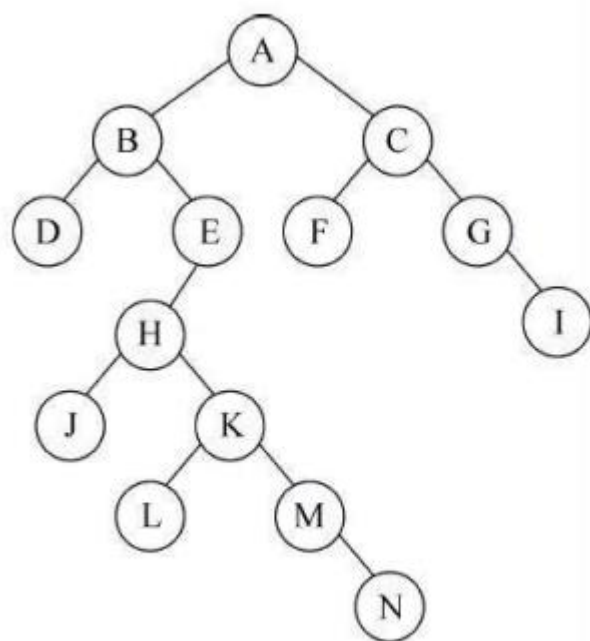


图 7.34 一棵二叉树

4.任务分析及算法设计思路

4.1 任务分析

二叉树采用**二叉链表**存储，每个结点包含：

- data: 结点数据（字符）
- lchild: 指向左孩子的指针
- rchild: 指向右孩子的指针

本实验关键点是如何把括号表示串转换成二叉链结构。括号表示串表达规则：

A(B,C) 表示 A 的左子树根为 B，右子树根为 C

逗号，用于区分左右子树

空子树可省略或表现为 (... , ...) 中某一侧为空

因此，创建算法需要在扫描字符串时能够知道：

新结点是个结点的左孩子还是右孩子

当前正在处理哪一个“父结点”

为此采用栈辅助建树：

遇到 '('：说明后面要建立该结点的孩子，把当前结点入栈作为父结点；遇到 ','：

切换到建立右孩子；遇到 ')'：孩子建立结束，父结点出栈回退；遇到字母：创建

新结点，并按当前方向挂到栈顶父结点的左/右孩子上

4.2 数据结构设计

(1) 二叉链结点结构:

```
typedef struct node {  
    ElemType data;  
    struct node *lchild;  
    struct node *rchild;  
} BTreeNode;
```

(2) 建树辅助栈:

- BTreeNode *St[MaxSize]: 存放路径上的祖先结点指针
- top: 栈顶指针
- k: 标记当前新结点应作为“左孩子(1)”还是“右孩子(2)”
- j: 扫描括号表示串的下标

(3) 功能函数所用指针:

- b: 二叉树根指针
- p: 查找目标结点指针
- lp/rp: 目标结点左右孩子指针

4.3 算法基本流程

(1) 由括号表示串创建二叉树 CreateBTree

初始化: b=NULL, top=-1, k=0

逐字符扫描 str:

若 ch == '(' : 父结点入栈, k=1 (下一结点是左孩子)

若 ch == ',' : k=2 (下一结点是右孩子)

若 ch == ')' : 出栈 (回到上层父结点)

若 ch 是数据字符:

 创建新结点 p

 若 b==NULL: b=p (p 为根)

 否则:

 若 k==1: St[top]->lchild=p

若 $k==2$: $St[top] \rightarrow rchild = p$

继续扫描直到结束

(2) 输出二叉树 DispBTree

递归输出:

- 输出根
- 若存在孩子则输出 (左子树 , 右子树) (右子树为空时可省略逗号输出规则由代码控制)

(3) 查找结点 FindNode

递归:

- 根为空返回空
- 根为目标返回根
- 否则先去左子树找, 再去右子树找

5.程序结构及代码解析

5.1 程序结构

btree.cpp: 实现二叉树基本运算

```
#include <stdio.h>
#include <stdlib.h>

#define MaxSize 100
typedef char ElemType;          // 结点数据类型

// 二叉链结点结构定义
typedef struct node {
    ElemType data;              // 数据域
    struct node *lchild;        // 指向左孩子
    struct node *rchild;        // 指向右孩子
} BTreeNode;

void CreateBTree(BTreeNode *&b, const char *str) {
    BTreeNode *St[MaxSize], *p = NULL;
    int top = -1, k = 0, j = 0; // top 为栈顶指针, k 表示孩子方向, j 为字符串下标
    char ch;
```

```

b = NULL;
ch = str[j];

while (ch != '\0') {
    switch (ch) {
        case '(':
            St[++top] = p;
            k = 1;
            break;
        case ')':
            --top;
            break;
        case ',':
            k = 2;
            break;
        default:

            p = (BTNode*)malloc(sizeof(BTNode));
            p->data = ch;
            p->lchild = p->rchild = NULL;

            if (b == NULL) {
                b = p;
            } else {

                if (k == 1) St[top]->lchild = p;
                else if (k == 2) St[top]->rchild = p;
            }
            break;
    }
    ch = str[++j];
}

//销毁二叉树

void DestroyBTree(BTNode *&b) {
    if (b != NULL) {
        DestroyBTree(b->lchild);    // 释放左子树
        DestroyBTree(b->rchild);    // 释放右子树
        free(b);                    // 释放根结点
        b = NULL;
    }
}

```

```

    }
}

BTNode* FindNode(BTNode *b, ElemType x) {
    BTNode *p;
    if (b == NULL) return NULL;        // 空树
    if (b->data == x) return b;         // 找到结点
    p = FindNode(b->lchild, x);         // 在左子树中查找
    if (p != NULL) return p;
    return FindNode(b->rchild, x);      // 在右子树中查找
}

BTNode* LchildNode(BTNode *p) {
    if (p == NULL) return NULL;
    return p->lchild;
}

BTNode* RchildNode(BTNode *p) {
    if (p == NULL) return NULL;
    return p->rchild;
}

int BTHight(BTNode *b) {
    int lh, rh;
    if (b == NULL) return 0;           // 空树高度为 0
    lh = BTHight(b->lchild);           // 左子树高度
    rh = BTHight(b->rchild);           // 右子树高度
    return (lh > rh) ? (lh + 1) : (rh + 1);
}

void DispBTree(BTNode *b) {
    if (b != NULL) {
        printf("%c", b->data);         // 输出根结点
        if (b->lchild != NULL || b->rchild != NULL) {
            printf("(");
            DispBTree(b->lchild);       // 输出左子树
            if (b->rchild != NULL) printf(",");
            DispBTree(b->rchild);       // 输出右子树
            printf(")");
        }
    }
}

```

```

    }
}
}

```

exp7-1.cpp: 调用 btree.cpp 完成实验要求功能

```

#include "btree.cpp"

int main() {
    BTreeNode *b = NULL, *p = NULL, *lp = NULL, *rp = NULL;

    printf("二叉树的基本运算如下:\n");
    printf("(1) 创建二叉树\n");
    CreateBTree(b, "A(B(D,E(H(J,K(L,M(,N))))),C(F,G(,I)))");

    printf("(2) 输出二叉树:");
    DispBTree(b);
    printf("\n");

    printf("(3) H 结点:");
    p = FindNode(b, 'H');
    if (p == NULL) {
        printf("树中不存在 H 结点\n");
    } else {
        lp = LchildNode(p);
        if (lp != NULL) printf("左孩子为%c ", lp->data);
        else printf("无左孩子 ");

        rp = RchildNode(p);
        if (rp != NULL) printf("右孩子为%c ", rp->data);
        else printf("无右孩子 ");
        printf("\n");
    }

    printf("(4) 二叉树 b 的高度:%d\n", BTreeHeight(b));

    printf("(5) 释放二叉树 b\n");
    DestroyBTree(b);

    return 0;
}

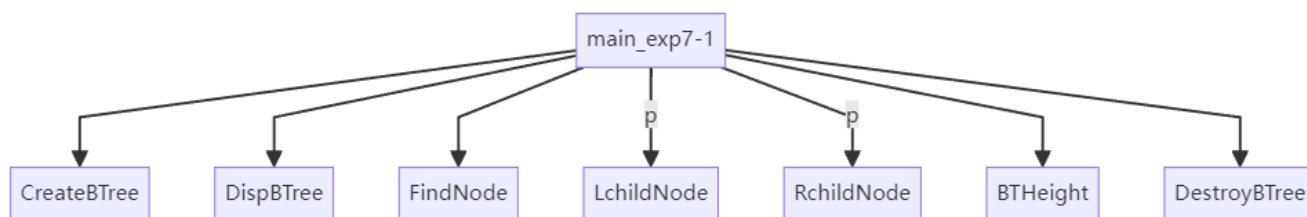
```

5.2 函数调用关系图


```

exp7-1.cpp
├── main()
│   ├── CreateBTree(b, str)      // 创建二叉树
│   ├── DispBTree(b)            // 输出二叉树
│   ├── FindNode(b, 'H') ──> p  // 查找结点 H
│   │   ├── LchildNode(p) ──> lp // 取左孩子
│   │   └── RchildNode(p) ──> rp // 取右孩子
│   ├── BTHeight(b)             // 求二叉树高度
│   └── DestroyBTree(b)         // 释放二叉树

```



6. 实验测试数据和实验结果分析

6.1 测试数据

- 二叉树括号表示串：
A(B(D, E(H(J, K(L, M(N))))), C(F, G(I))))
- 查找结点：H

6.2 预期结果分析

1) 输出二叉树时，应与输入括号串对应（结构一致）。

2) H 的左右孩子：

从括号串片段 H(J, K(...)) 可知：

左孩子为 J

右孩子为 K

3) 树的高度应由递归计算得到（从根到最深叶结点路径长度的结点数）。本树最深路径在 A→B→E→H→K→M→N（注意 M 的右孩子为 N），因此高度应为 7。

4) 销毁后不应发生内存泄漏或非法访问。

6.3 程序运行结果

```
D:\Users\86187\Desktop\8207 × + v
二叉树的基本运算如下：
(1)创建二叉树
(2)输出二叉树:A(B(D,E(H(J,K(L,M(,N))))),C(F,G(,I)))
(3)H结点:左孩子为J 右孩子为K
(4)二叉树b的高度:7
(5)释放二叉树b

-----
Process exited after 0.4333 seconds with return value 0
请按任意键继续. . . |
```

7.实验总结与体会

实验收获

通过本实验，我对二叉树采用二叉链存储时的结构特点有了更清晰的认识：每个结点只需要保存数据域以及左右孩子指针，就能完整表达一棵树的层次关系。相比顺序存储，链式结构在表示不规则形态的树时更自然，也更便于进行递归处理。通过亲手实现结点结构体、指针连接与遍历过程，我对“树是由根结点和左右子树递归构成”的概念理解得更扎实。

在由括号表示串创建二叉树的过程中，我体会到“栈 + 扫描”的方法非常实用。括号表示串本质上是层次结构的线性化描述，直接按字符顺序读取时会不断进入更深层的子树，栈正好可以保存“当前父结点路径”，在遇到) 时快速回到上层父结点，保证挂接孩子时不会错位。这让我进一步理解了栈“后进先出”的特性与树结构回退过程之间的对应关系。此外，查找结点、输出括号表示串、求高度以及销毁二叉树等操作都体现了递归在树结构上的优势：递归写法与树的定义一致，代码逻辑清晰，容易验证正确性。尤其是求高度与销毁操作，分别对应“自底向上汇总信息”和“后序释放资源”的思想，让我对二叉树常见算法模式有了整体把握。

遇到的问题及解决办法

建树过程中父结点定位困难,当括号嵌套层数较深时，如果只靠当前指针移动，很难判断新结点应该挂到哪一个父结点上，容易出现结点连接错误。使用指针栈保

存祖先结点路径。遇到 (将当前结点入栈，遇到) 出栈回退，这样栈顶始终表示当前正在建立孩子的父结点，父子关系不会混乱。

左右孩子挂接方向容易出错:括号串中逗号 , 是左右子树的分界点，如果忽略逗号或未及时切换方向，新结点可能被错误地连续挂到左孩子位置，导致生成的树结构与原串不一致。设置方向标记 k，用 k=1 表示接左孩子，k=2 表示接右孩子。遇到 (设置 k=1，遇到 , 设置 k=2，创建新结点时根据 k 选择挂接到 lchild 或 rchild。

任务 3（选做）

求二叉树中从根结点到叶子结点的路径，实验内容见主教材（第六版）第 244 页的实验题 7。

1. 实验项目名称

求二叉树中从根结点到叶子结点的路径

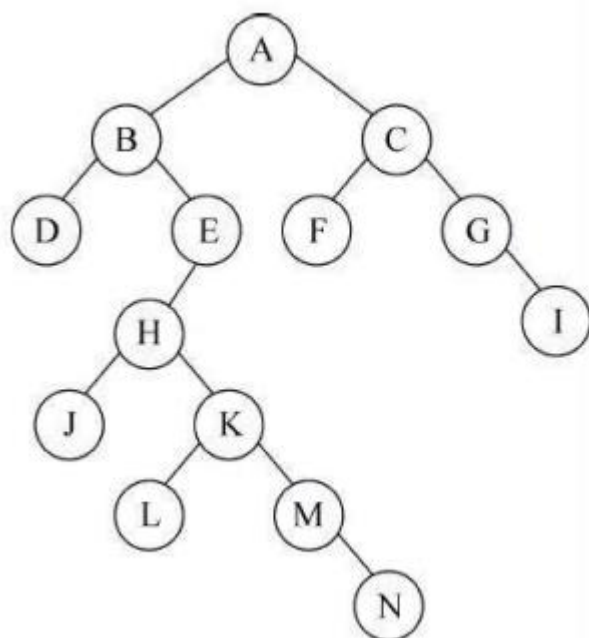
2. 实验目的

掌握二叉树遍历算法的应用，熟练使用先序、中序、后序递归和非递归遍历算法以及层次遍历算法进行二叉树问题的求解。

3. 实验任务描述

编写一个程序 exp7-7.cpp 实现以下功能，并对图 7.33 所示的二叉树进行验证。

- （1）采用先序遍历方法输出所有从叶子结点到根结点的逆路径。
- （2）采用先序遍历方法输出第一条最长的逆路径。
- （3）采用后序非递归遍历方法输出所有从叶子结点到根结点的逆路径。
- （4）采用层次遍历方法输出所有从叶子结点到根结点的逆路径。



4.任务分析及算法设计思路

4.1 任务分析

本题要输出的是叶子→根的逆路径。对每个叶子结点，只要能得到“根→叶”的路径序列，就能反向打印得到逆路径。

不同遍历方式的关键差异在于：

- 递归遍历时，系统调用栈天然具备“回溯”能力，容易维护当前路径；
- 非递归后序遍历需要显式栈来模拟访问顺序，同时栈也能保存“根→当前结点”的路径；
- 层次遍历是队列结构，不保存祖先链，因此需要额外记录“父结点信息”，才能从叶子回溯到根。

4.2 数据结构设计

1) 二叉链结点：

data: 字符数据

lchild / rchild: 左右孩子指针

2) 先序递归路径数组：

- path[]: 保存根→当前结点（祖先链）
- pathlen: 当前路径长度（祖先个数）
- longpath[] / longpathlen: 保存第一条最长路径（根→叶）

3) 后序非递归辅助栈：

- st[] + top: 模拟遍历并保存根到当前结点路径
- r: 指向“刚访问过的结点”，用于判断右子树是否已处理（后序非递归的关键）

4) 层次遍历队列（带父指针下标）：

```
typedef struct {  
    BTreeNode *node;  
    int parent;    // 父结点在队列中的下标  
} QNode;
```

叶子结点出队时，通过 parent 反向追溯并输出叶→根。

4.3 算法基本流程

(1) 先序递归输出所有逆路径 AllPath1

遇到非叶子结点：把当前结点加入 path，递归左子树再递归右子树（先序）。

遇到叶子结点：直接输出“叶子→(祖先逆序)→根”。

(2) 先序递归求第一条最长逆路径 LongPath1

在递归过程中始终维护“根→当前结点”的路径 path。

当访问到叶子结点：如果当前长度 pathlen 大于 longpathlen，就更新最优记录。

最终把 longpath 反向输出得到“叶→根”的第一条最长逆路径。

因为是先序“先左后右”，所以当多条最长路径存在时，会保留第一次遇到的那条（满足“第一条最长”）。

(3) 后序非递归输出所有逆路径 AllPath2

用栈模拟后序遍历：不断向左入栈；查看栈顶结点，若其右子树为空或已访问，则访问该结点并出栈。

当访问到叶子结点时，栈中刚好保存了“根→叶”的路径，按栈从顶到底即可输出“叶→根”。

(4) 层次遍历输出所有逆路径 AllPath3

根结点入队，parent=-1。

出队一个结点：若它是叶子结点，则沿 parent 下标追溯到根并输出“叶→根”。

将其左右孩子入队，同时记录孩子结点的 parent=当前结点下标。

5. 程序结构及代码解析

本实验一个源文件（exp7-7.cpp），主要模块：

建树与释放：CreateBTree、DestroyBTree

先序递归：AllPath1（所有逆路径）、LongPath1（第一条最长逆路径）

后序非递归：AllPath2（所有逆路径）

层次遍历：AllPath3（所有逆路径）

main：创建树 → 依次调用四种方法输出 → 释放树

```
#include <stdio.h>
#include <stdlib.h>
```

```

#define MaxSize 100
typedef char ElemType;

typedef struct node {
    ElemType data;
    struct node *lchild;
    struct node *rchild;
} BTreeNode;

void CreateBTree(BTreeNode *&b, const char *str) {
    BTreeNode *St[MaxSize], *p = NULL;
    int top = -1, k = 0, j = 0;
    char ch;

    b = NULL;
    ch = str[j];

    while (ch != '\0') {
        switch (ch) {
            case '(':
                St[++top] = p;    // 当前结点入栈
                k = 1;           // 接下来处理左孩子
                break;
            case ')':
                --top;           // 子树结束出栈
                break;
            case ',':
                k = 2;           // 转向处理右孩子
                break;
            default:
                p = (BTreeNode*)malloc(sizeof(BTreeNode));
                p->data = ch;
                p->lchild = p->rchild = NULL;

                if (b == NULL) {
                    b = p;       // 根结点
                } else {
                    if (k == 1) St[top]->lchild = p;
                    else if (k == 2) St[top]->rchild = p;
                }
                break;
        }
        ch = str[++j];
    }
}

```

```

        ch = str[++j];
    }
}

// 销毁二叉树
void DestroyBTree(BTNode *&b) {
    if (b != NULL) {
        DestroyBTree(b->lchild);
        DestroyBTree(b->rchild);
        free(b);
        b = NULL;
    }
}

void AllPath1(BTNode *b, ElemType path[], int pathlen) {
    if (b != NULL) {
        // 如果 b 是叶子结点: 直接输出 “叶->...->根”
        if (b->lchild == NULL && b->rchild == NULL) {
            printf(" %c 到根结点逆路径: %c->", b->data, b->data);
            for (int i = pathlen - 1; i > 0; --i)
                printf("%c->", path[i]);
            if (pathlen > 0) printf("%c\n", path[0]);
            else printf("\n"); // 只有根一个结点时
        } else {
            // 非叶子: 把当前结点压入祖先路径, 递归扫描左右子树
            path[pathlen] = b->data;
            pathlen++;
            AllPath1(b->lchild, path, pathlen);
            AllPath1(b->rchild, path, pathlen);
        }
    }
}

void LongPath1(BTNode *b, ElemType path[], int pathlen,
               ElemType longpath[], int &longpathlen) {
    if (b == NULL) return;

    // 把当前结点加入路径 (含当前结点)
    path[pathlen] = b->data;
    pathlen++;

    // 叶子: 比较并更新 “第一条最长路径”
    if (b->lchild == NULL && b->rchild == NULL) {

```



```

        if (pathlen > longpathlen) {
            for (int i = 0; i < pathlen; ++i)
                longpath[i] = path[i];
            longpathlen = pathlen;
        }
        return;
    }

    // 先序：先左后右
    LongPath1(b->lchild, path, pathlen, longpath, longpathlen);
    LongPath1(b->rchild, path, pathlen, longpath, longpathlen);
}

void AllPath2(BTNode *b) {
    BTNode *st[MaxSize]; // 栈保存“根->...->当前结点”路径
    int top = -1;
    BTNode *p = b;
    BTNode *r = NULL;    // r 指向刚刚访问过的结点

    while (p != NULL || top != -1) {
        // 沿左分支一路入栈
        while (p != NULL) {
            st[++top] = p;
            p = p->lchild;
        }

        // 查看栈顶
        p = st[top];

        // 若右子树不存在或已访问过，则访问该结点（后序“根”）
        if (p->rchild == NULL || p->rchild == r) {

            // 如果访问到的是叶子结点：输出叶->根逆路径
            if (p->lchild == NULL && p->rchild == NULL) {
                printf(" %c 到根结点逆路径: %c->", p->data, p->data);
                for (int i = top - 1; i > 0; --i)
                    printf("%c->", st[i]->data);
                if (top > 0) printf("%c\n", st[0]->data);
                else printf("\n");
            }

            // 出栈访问
            top--;

```

```

        r = p;
        p = NULL; // 关键：防止重复走左路
    } else {
        // 转向处理右子树
        p = p->rchild;
    }
}
}

typedef struct {
    BTreeNode *node;    // 队列中保存的结点指针
    int parent;         // 该结点在队列中的父结点下标（用于回溯到根）
} QNode;

void AllPath3(BTreeNode *b) {
    if (b == NULL) return;

    QNode Qu[MaxSize];
    int front = -1, rear = -1;

    // 根结点入队，父下标为 -1
    Qu[++rear].node = b;
    Qu[rear].parent = -1;

    while (front != rear) {
        front++;
        BTreeNode *cur = Qu[front].node;

        // 若为叶子：沿 parent 回溯输出叶->根逆路径
        if (cur->lchild == NULL && cur->rchild == NULL) {
            printf(" %c 到根结点逆路径: ", cur->data);
            int p = front;
            while (Qu[p].parent != -1) {
                printf("%c->", Qu[p].node->data);
                p = Qu[p].parent;
            }
            printf("%c\n", Qu[p].node->data);
        }

        // 左孩子入队
        if (cur->lchild != NULL) {
            Qu[++rear].node = cur->lchild;
            Qu[rear].parent = front;
        }
    }
}

```

```

    }
    // 右孩子入队
    if (cur->rchild != NULL) {
        Qu[++rear].node = cur->rchild;
        Qu[rear].parent = front;
    }
}
}

int main() {
    BTreeNode *b = NULL;
    ElemType path[MaxSize], longpath[MaxSize];
    int longpathlen = 0;

    CreateBTree(b, "A(B(D,E(H(J,K(L,M(,N))))),C(F,G(,I))))");
    printf("A(B(D,E(H(J,K(L,M(,N))))),C(F,G(,I))))\n\n");

    printf("叶结点到根结点逆路径如下: \n");
    AllPath1(b, path, 0);

    // 第一条最长逆路径 (先序)
    LongPath1(b, path, 0, longpath, longpathlen);
    printf("第一条最长逆路径长度: %d\n", longpathlen);
    printf("第一条最长逆路径: ");
    for (int i = longpathlen - 1; i >= 0; --i) {
        printf("%c", longpath[i]);
        if (i != 0) printf("->");
    }
    printf("\n");

    printf("\n 后序非递归遍历方法: \n");
    AllPath2(b);

    printf("\n 层次遍历方法: \n");
    AllPath3(b);

    DestroyBTree(b);
    return 0;
}

```

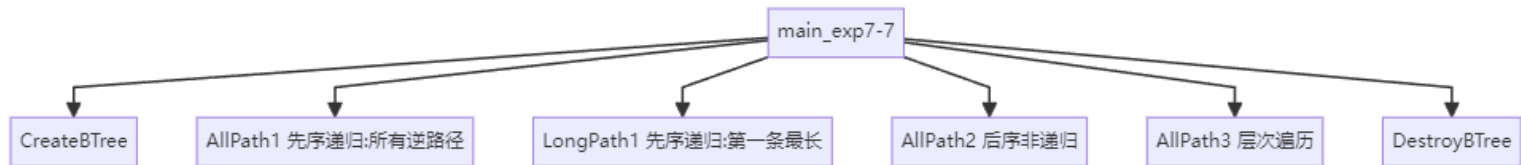
函数调用关系图

main()

```

|—— CreateBTree(b, str)
|—— AllPath1(b, path, 0)           // 先序递归：所有逆路径
|—— LongPath1(b, path, 0, longpath, longpathlen)
|    |—— (main 中反向输出 longpath 得到“叶->根”最长逆路径)
|—— AllPath2(b)                   // 后序非递归：所有逆路径
|—— AllPath3(b)                   // 层次遍历：所有逆路径
|—— DestroyBTree(b)

```



6.实验测试数据和实验结果分析

6.1 测试数据

二叉树括号串：

A(B(D, E(H(J, K(L, M(, N))))), C(F, G(, I))))

该树的叶子结点为：D、J、L、N、F、I

6.2 程序运行结果

```

D:\Users\86187\Desktop\8207 x + v
A(B(D,E(H(J,K(L,M(,N))))),C(F,G(,I))))
叶结点到根结点逆路径如下：
D到根结点逆路径：D->B->A
J到根结点逆路径：J->H->E->B->A
L到根结点逆路径：L->K->H->E->B->A
N到根结点逆路径：N->M->K->H->E->B->A
F到根结点逆路径：F->C->A
I到根结点逆路径：I->G->C->A
第一条最长逆路径长度：7
第一条最长逆路径：N->M->K->H->E->B->A
后序非递归遍历方法：
D到根结点逆路径：D->B->A
J到根结点逆路径：J->H->E->B->A
L到根结点逆路径：L->K->H->E->B->A
N到根结点逆路径：N->M->K->H->E->B->A
F到根结点逆路径：F->C->A
I到根结点逆路径：I->G->C->A
层次遍历方法：
D到根结点逆路径：D->B->A
F到根结点逆路径：F->C->A
I到根结点逆路径：I->G->C->A
J到根结点逆路径：J->H->E->B->A
L到根结点逆路径：L->K->H->E->B->A
N到根结点逆路径：N->M->K->H->E->B->A
-----
Process exited after 0.48 seconds with return value 0
请按任意键继续. . .

```

7.实验总结与体会

实验收获

通过本实验，我进一步理解了“遍历 = 枚举结点的一种顺序”，而“路径输出”本质是对遍历过程中祖先信息的维护。先序递归方法实现最直观：递归深入时把结点加入路径数组，回溯时自动恢复现场，到叶子就能直接输出结果。对“第一条最长逆路径”的求解也体现了典型的“遍历过程中维护最优解”的思路：每次到叶子进行一次比较更新，最终得到最优路径。

同时，我也加深了对非递归遍历与层次遍历特点的认识。后序非递归遍历虽然写法更复杂，但栈在其中不仅用于控制访问顺序，也自然保存了从根到当前结点的路径；而层次遍历由于使用队列，不具备回溯能力，因此必须额外记录父结点信息才能实现叶到根的追溯输出。这让我更清楚地看到：不同遍历方式解决同一问题时，核心差别往往在“如何保存与恢复路径信息”。

遇到的问题及解决办法

层次遍历无法直接得到路径：队列只保证访问顺序，不保存祖先链。队列元素增加 `parent` 下标，叶子出队时沿 `parent` 追溯输出可解决。

后序非递归容易重复走右子树：如果没有访问标记，会反复进入右分支。解决：使用指针 `r` 记录“刚访问过的结点”，当栈顶结点的右孩子等于 `r` 时说明右子树已处理，可以安全访问栈顶并出栈。

最长路径的“第一条”要求：如果更新条件写成 `>=`，会把后出现的同长度路径覆盖掉更新条件使用 `>`，只在更长时更新，从而保留先序遍历遇到的第一条最长路径。

实验三：图的有关算法设计

要求：有 3 个任务，一个必做实验，两个选做实验（任选其一）

任务 1（必做）

求有向图的简单路径，实验内容见主教材（第六版）第 309 页实验题 10，要求：创建一个图，并输出求得的路径。

1.实验项目名称

求有向图的简单路径

2.实验目的

掌握深度优先遍历和广度优先遍历算法在求解图路径问题中的应用。

3.实验任务描述

编写一个程序 exp8-10.cpp，设计相关算法完成以下功能。

- （1）输出如图 8.56 所示的有向图 G 中从顶点 5 到顶点 2 的所有简单路径。
- （2）输出如图 8.56 所示的有向图 G 中从顶点 5 到顶点 2 的所有长度为 3 的简单路径。
- （3）输出如图 8.56 所示的有向图 G 中从顶点 5 到顶点 2 的最短路径。

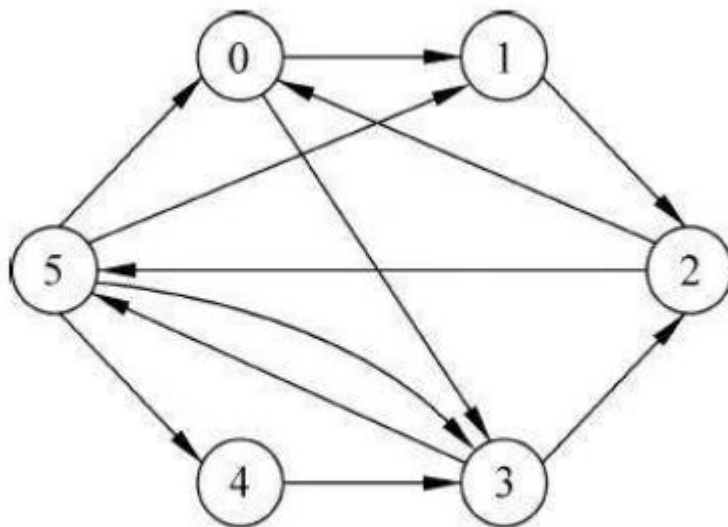


图 8.56 一个有向图

4.任务分析及算法设计思路

4.1 任务分析

有向图路径问题的核心在于：简单路径要求路径中顶点不重复，否则会形成回路导致重复搜索甚至死循环。要输出“所有路径”，不能像只找一条路径那样到达终点就结束，而是需要继续回溯搜索。“长度为 3 的路径”属于约束枚举，需要在 DFS 过程中控制深度并剪枝。“最短路径”可用 BFS（无权或等权图）求从起点到各点的最短边数距离，再输出对应路径。

4.2 数据结构设计

(1) 邻接表结构（有向图）：

- ArcNode：边结点（邻接点编号 adjvex、权值 w、next 指针）
- VNode：顶点结点（指向第一条边 firstarc）
- AdjGraph：图结构（顶点数组、顶点数 n、边数 e）

(2) 访问标记与路径数组：

- visited[]：DFS 中标记当前路径已访问的顶点（保证简单路径）
- path[]：保存当前 DFS 的路径序列（从起点到当前点）

(3) 最短路径辅助结构：

- dist[]：BFS 计算最短边数距离
- pred[y][] 与 predCnt[y]：记录所有能得到最短距离的前驱结点（用于输出所有最短路径）
- q[]：BFS 队列
- DFSPrintAllShortest()：对“前驱图”进行回溯 DFS 输出最短路径

4.3 算法基本流程

1) 输出所有简单路径（DFS 回溯）

```
PathAll1(u, v, path, d):  
    d++, path[d] = u  
    visited[u] = 1  
    if u == v:  
        输出 path[0..d]  
        visited[u] = 0  
        return  
    for u 的每个邻接点 w:  
        if visited[w] == 0:
```

```

        PathAll1(w, v, path, d)
    visited[u] = 0    // 回溯恢复

```

(2) 输出所有长度为 11 的简单路径（带剪枝 DFS）

```

PathAll2(u, v, 11, path, d):
    visited[u] = 1
    d++, path[d] = u
    if u == v 且 d == 11:
        输出 path[0..d]
        visited[u] = 0
        return
    if d >= 11:
        visited[u] = 0
        return
    for u 的每个邻接点 w:
        if visited[w] == 0:
            PathAll2(w, v, 11, path, d)
    visited[u] = 0

```

这里的 d 表示从起点到当前结点经历的**边数**（主函数从 $d=-1$ 开始传入，因此到起点后 $d=0$ ）。

(3) 最短路径（BFS + 记录前驱 + 回溯输出）

```

BFS(u):
    dist[] 初始化为 -1
    dist[u]=0, u 入队
    while 队列不空:
        出队 x
        for x 的邻接点 y:
            if dist[y] == -1:
                dist[y] = dist[x] + 1
                pred[y] = {x}
                y 入队
            else if dist[y] == dist[x] + 1:
                pred[y] 追加 x    // 记录另一条最短前驱

```

输出最短路径:

从终点 v 沿 $pred$ 回溯到 u (DFS), 输出 $u \rightarrow \dots \rightarrow v$

5. 程序结构及代码解析

5.1 程序结构

图的创建: `CreateAdjG()` (由邻接矩阵创建邻接表)

图的显示: DispEdges() (输出边集, 便于核对图结构)

DFS 全部简单路径: PathAll1()

DFS 固定长度简单路径: PathAll2()

BFS 最短路径: AllShortPaths() + DFSPrintAllShortest()

释放图: DestroyAdjG()

main(): 建图 → 输出边集 → 输出(1)(2)(3)结果 → 释放

```
#include <stdio.h>
#include <stdlib.h>

#define MAXV 20
#define MAXSIZE 100

typedef struct ArcNode {
    int adjvex;           // 邻接点
    int w;                // 权值
    struct ArcNode *nextarc; // 下一条边
} ArcNode;

typedef struct VNode {
    ArcNode *firstarc;    // 顶点的第一条边
} VNode;

typedef struct AdjGraph {
    VNode adjlist[MAXV];  // 邻接表
    int n, e;             // 顶点数、边数(弧数)
} AdjGraph;

int visited[MAXV];

void CreateAdjG(AdjGraph *G, int A[][MAXV], int n) {
    int i, j;
    ArcNode *p;

    G->n = n;
    G->e = 0;
    for (i = 0; i < n; i++)
        G->adjlist[i].firstarc = NULL;

    for (i = 0; i < n; i++) {
        for (j = n - 1; j >= 0; j--) { // 从大到小扫描, 配合头插
```

```

        if (A[i][j] != 0) {
            p = (ArcNode*)malloc(sizeof(ArcNode));
            if (!p) {
                fprintf(stderr, "malloc failed\n");
                exit(1);
            }
            p->adjvex = j;
            p->w = A[i][j];
            p->nextarc = G->adjlist[i].firstarc;
            G->adjlist[i].firstarc = p;
            G->e++;
        }
    }
}

void DispEdges(AdjGraph *G) {
    int i;
    ArcNode *p;
    printf("图 G 的边集: \n");
    for (i = 0; i < G->n; i++) {
        p = G->adjlist[i].firstarc;
        while (p != NULL) {
            printf("%d -> %d (w=%d)\n", i, p->adjvex, p->w);
            p = p->nextarc;
        }
    }
}

void PathAll1(AdjGraph *G, int u, int v, int path[], int d) {
    ArcNode *p;
    int j, w;

    d++;
    path[d] = u;
    visited[u] = 1;

    if (u == v) {
        for (j = 0; j <= d; j++) printf("%3d", path[j]);
        printf("\n");
        visited[u] = 0;
        return;
    }
}

```

```

    p = G->adjlist[u].firstarc;
    while (p != NULL) {
        w = p->adjvex;
        if (visited[w] == 0)
            PathAll1(G, w, v, path, d);
        p = p->nextarc;
    }

    visited[u] = 0;
}

// 输出从u到v的所有长度为l1(边数)的简单路径
void PathAll2(AdjGraph *G, int u, int v, int l1, int path[], int d) {
    ArcNode *p;
    int j, w;

    visited[u] = 1;
    d++;
    path[d] = u;

    if (u == v && d == l1) {
        for (j = 0; j <= d; j++) printf("%3d", path[j]);
        printf("\n");
        visited[u] = 0;
        return;
    }

    // 剪枝: 边数已经超过l1, 或已经到l1但还没到v, 都没必要再往下走
    if (d >= l1) {
        visited[u] = 0;
        return;
    }

    p = G->adjlist[u].firstarc;
    while (p != NULL) {
        w = p->adjvex;
        if (visited[w] == 0)
            PathAll2(G, w, v, l1, path, d);
        p = p->nextarc;
    }

    visited[u] = 0;
}

```

```

static void DFSPrintAllShortest(int s, int cur,
                                int pred[][MAXV], int predCnt[],
                                int path[], int len) {

    int i, k;
    path[len] = cur;

    if (cur == s) {
        // path 是 v->...->s, 逆序打印就是 s->...->v
        for (k = len; k >= 0; k--) printf("%3d", path[k]);
        printf("\n");
        return;
    }

    for (i = 0; i < predCnt[cur]; i++) {
        DFSPrintAllShortest(s, pred[cur][i], pred, predCnt, path, len + 1);
    }
}

void AllShortPaths(AdjGraph *G, int u, int v) {
    int dist[MAXV];
    int pred[MAXV][MAXV];
    int predCnt[MAXV];
    int q[MAXV];
    int front = 0, rear = 0;
    int i;
    ArcNode *p;

    for (i = 0; i < G->n; i++) {
        dist[i] = -1;
        predCnt[i] = 0;
    }

    dist[u] = 0;
    q[rear++] = u;

    // BFS: 求 dist, 并记录所有最短前驱
    while (front < rear) {
        int x = q[front++];

        p = G->adjlist[x].firstarc;
        while (p != NULL) {
            int y = p->adjvex;

```

```

        if (dist[y] == -1) {
            dist[y] = dist[x] + 1;
            predCnt[y] = 0;
            pred[y][predCnt[y]++] = x;
            q[rear++] = y;
        } else if (dist[y] == dist[x] + 1) {
            if (predCnt[y] < MAXV) { // 防止越界
                pred[y][predCnt[y]++] = x;
            }
        }

        p = p->nextarc;
    }
}

if (dist[v] == -1) {
    printf("无路径\n");
    return;
}

{
    int path[MAXV];
    DFSPrintAllShortest(u, v, pred, predCnt, path, 0);
}
}

void DestroyAdjG(AdjGraph *G) {
    int i;
    ArcNode *p, *q;
    for (i = 0; i < G->n; i++) {
        p = G->adjlist[i].firstarc;
        while (p != NULL) {
            q = p->nextarc;
            free(p);
            p = q;
        }
        G->adjlist[i].firstarc = NULL;
    }
}

int main() {
    int i;

```

```

int u = 5, v = 2, l1 = 3;
int path[MAXV];
AdjGraph G;

int A[MAXV][MAXV] = {0};
int n = 6;

A[0][1] = 1; A[0][3] = 1;
A[1][2] = 1;
A[2][0] = 1; A[2][5] = 1;
A[3][2] = 1; A[3][5] = 1;
A[4][3] = 1;
A[5][0] = 1; A[5][1] = 1; A[5][3] = 1; A[5][4] = 1;

CreateAdjG(&G, A, n);

DispEdges(&G);

printf("\n(1)从顶点%d到%d的所有路径: \n", u, v);
for (i = 0; i < MAXV; i++) visited[i] = 0;
PathAll1(&G, u, v, path, -1);

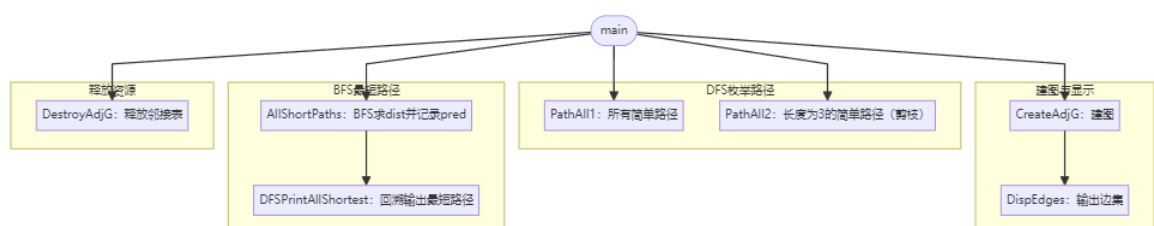
printf("\n(2)从顶点%d到%d的所有长度为%d(边数)的路径: \n", u, v, l1);
for (i = 0; i < MAXV; i++) visited[i] = 0;
PathAll2(&G, u, v, l1, path, -1);

printf("\n(3)从顶点%d到%d的所有最短路径: \n", u, v);
AllShortPaths(&G, u, v);

DestroyAdjG(&G);
return 0;
}

```

5.2 函数调用关系图



6. 实验测试数据和实验结果分析

6.1 测试数据

程序中建立的图（顶点 $0 \sim 5$ ），边为：

- $0 \rightarrow 1, 0 \rightarrow 3$
- $1 \rightarrow 2$
- $2 \rightarrow 0, 2 \rightarrow 5$
- $3 \rightarrow 2, 3 \rightarrow 5$
- $4 \rightarrow 3$
- $5 \rightarrow 0, 5 \rightarrow 1, 5 \rightarrow 3, 5 \rightarrow 4$

6.2 程序运行结果

```
D:\Users\86187\Desktop\8207 × + v
图G的边集：
0 -> 1 (w=1)
0 -> 3 (w=1)
1 -> 2 (w=1)
2 -> 0 (w=1)
2 -> 5 (w=1)
3 -> 2 (w=1)
3 -> 5 (w=1)
4 -> 3 (w=1)
5 -> 0 (w=1)
5 -> 1 (w=1)
5 -> 3 (w=1)
5 -> 4 (w=1)

(1)从顶点5到2的所有路径：
5 0 1 2
5 0 3 2
5 1 2
5 3 2
5 4 3 2

(2)从顶点5到2的所有长度为3(边数)的路径：
5 0 1 2
5 0 3 2
5 4 3 2

(3)从顶点5到2的所有最短路径：
5 1 2
5 3 2

-----
Process exited after 0.4666 seconds with return value 0
请按任意键继续. . . |
```

7.实验总结与体会

实验收获

通过本实验，我对 DFS 枚举路径和 BFS 求最短路径这两类典型图算法的适用场景更加清晰：当需要输出所有可能路径时，DFS 的回溯机制非常合适；而当只关心最短距离（按边数）时，BFS 能以层次扩展的方式保证第一次到达即为最短。尤其是对简单路径的处理，让我进一步理解了 visited 标记在图搜索中避免环路与重复访问的关键作用。

同时，固定长度路径的求解让我体会到剪枝的意义：在 DFS 过程中加入深度限制，能够减少无意义分支搜索，使程序更高效、更易控制输出结果范围。最短路径部分通过记录多个前驱并回溯输出，也让我认识到：最短路径不一定唯一，完整记录前驱能得到更全面的答案。

遇到的问题及解决办法

图中存在回路导致重复搜索或死循环风险，使用 visited[] 标记当前路径已访问顶点，回溯时恢复，确保枚举的是简单路径。

固定长度路径容易超深搜索，输出不符合“长度=3”的要求。用 d 记录当前边数，并在 $d \geq 11$ 时剪枝返回，且仅在 $u==v \ \&\& \ d==11$ 时输出。

最短路径可能不唯一，若只记录一个前驱会漏解。BFS 中对每个结点记录所有满足 $\text{dist}[y]==\text{dist}[x]+1$ 的前驱，最后回溯可输出所有最短路径。

任务 2（选做）

求解最短路径问题，实验内容：见主教材（第六版）第 311 页实验题 16。

1.实验项目名称

求解最短路径问题

2.实验目的

深入掌握 Dijkstra 算法在求解实际问题中的应用。

3.实验任务描述

编写一个程序 `exp8-16.cpp`，用来求解最短路径问题。假设有 n 个点， m 条无向边，每条边都有长度 d 和花费 p ，给定起点 s 和终点 t ，要求输出从起点到终点的最短距离及其花费。如果最短距离有多条路径线，则输出花费最少的。

测试数据存放在 `data16.txt` 文件中，第一行为 n, m ，点的编号是 $1 \sim n$ ；然后是 m 行，每行 4 个数 a, b, d, p ，表示 a 和 b 之间有一条边，且其长度为 d ，花费为 p ；最后一行是两个数 s, t ，表示起点 s 和终点 t 。输出结果有两个数，分别表示最短距离及其花费。

例如，`data16.txt` 的数据如下：

```
32
1256
2345
13
```

答案是 9,11，表示最短距离为 9，对应的花费是 11。

4.任务分析及算法设计思路

4.1 任务分析

本题是无向图最短路径，但每条边有两个属性：

- 距离（长度） d ：作为第一关键字，路径总距离最小；
- 花费 p ：当存在多条“总距离相同的最短路”时，选择总花费最小的那条。

因此算法要同时维护两套累计值：

- $\text{dist}[x]$: 从起点到 x 的最短距离
- $\text{cost}[x]$: 在 $\text{dist}[x]$ 最小时对应的最小花费

本题边权距离为非负数, 适合使用 **Dijkstra**。只需要在“松弛”时加入“距离相同则比较花费”的规则即可。

4.2 算法执行步骤

初始化:

```
dist[i] = D[s][i]
cost[i] = C[s][i]
S[i] = 0
dist[s]=0, cost[s]=0, S[s]=1
```

重复 $n-1$ 次:

```
从所有 S[i]==0 的点中找 dist 最小的 u
若 u 不存在 (不可达) 则退出
S[u]=1
for 每个未确定的 v:
    若 u->v 有边:
        d = dist[u] + D[u][v]
        c = cost[u] + C[u][v]
        if d < dist[v]:
            dist[v] = d
            cost[v] = c
        else if d == dist[v] 且 c < cost[v]:
            cost[v] = c
```

输出 $\text{dist}[t]$, $\text{cost}[t]$

要点:

- 先比距离, 再比花费, 保证输出满足题目规则。
- 若输入中同一对点有多条边: 保留更优的那条 (距离更短; 距离相同则花费更小)。

5.程序结构及代码解析

5.1 程序结构

exp8-16.cpp

main(): 读取数据→初始化矩阵→读边→读 s, t →调用 Dijkstra 输出结果

Dijkstra(): 按最短距离优先, 距离相同取最小花费的规则求解并输出

```
#include <stdio>
#include <stdlib>

const int MAXV = 1010;
const int INF = 0x3f3f3f3f;

int n, m;
int D[MAXV][MAXV], C[MAXV][MAXV]; // D: 距离(长度) C: 花费
int s, t;

// Dijkstra:
void Dijkstra() {
    int dist[MAXV], cost[MAXV];
    int S[MAXV];
    int mindist, u;

    // 初始化 dist / cost / S
    for (int i = 1; i <= n; i++) {
        dist[i] = D[s][i];
        cost[i] = C[s][i];
        S[i] = 0;
    }
    dist[s] = 0;
    cost[s] = 0;
    S[s] = 1;

    // 主循环: 执行 n-1 次
    for (int i = 1; i <= n - 1; i++) {
        mindist = INF;
        u = -1;

        // 在未加入 S 集合的点中, 找 dist 最小的点 u
        for (int j = 1; j <= n; j++) {
            if (S[j] == 0 && dist[j] < mindist) {
                mindist = dist[j];
                u = j;
            }
        }

        // 找不到可达点, 结束
        if (u == -1 || mindist == INF) break;

        // 将 u 加入 S 集合
```

```

    S[u] = 1;

    // 用 u 去松弛其他点
    for (int v = 1; v <= n; v++) {
        if (S[v] == 0 && D[u][v] < INF) {
            int d = dist[u] + D[u][v];
            int c = cost[u] + C[u][v];

            // 距离更短, 更新
            if (d < dist[v]) {
                dist[v] = d;
                cost[v] = c;
            }
            // 距离相同, 花费更小, 更新
            else if (d == dist[v] && c < cost[v]) {
                cost[v] = c;
            }
        }
    }
}

printf("\n 求解说明\n");

printf("规则: 优先最短距离; 若最短距离有多条路径, 则取花费最少的一条\n");
printf("起点: %d  终点: %d\n", s, t);

printf("\n 求解结果\n");
if (dist[t] >= INF) {
    printf("从 %d 到 %d 不存在可达路径! \n", s, t);
} else {
    printf("最短距离: %d\n", dist[t]);
    printf("对应花费: %d\n", cost[t]);
}
}

int main() {

    if (freopen("test3-2.txt", "r", stdin) == NULL) {
        printf("无法打开 .txt, 请确认文件与程序在同一目录.\n");
        return 0;
    }
}

```

```

// 读 n, m
if (scanf("%d%d", &n, &m) != 2) {
    printf("读取 n,m 失败, 请检查 .txt 第一行格式。\\n");
    return 0;
}

printf("读取数据说明\\n");
printf("\\n 数据格式: \\n");
printf("第一行: n m (点编号 1~n) \\n");
printf("接着 m 行: a b d p (无向边 a-b, 长度 d, 花费 p) \\n");
printf("最后一行: s t (起点 s, 终点 t) \\n");

printf("\\n 读取到: n = %d, m = %d\\n", n, m);

// 初始化矩阵为 INF (无边)
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= n; j++) {
        D[i][j] = INF;
        C[i][j] = INF;
    }
    D[i][i] = 0;
    C[i][i] = 0;
}

// 读取 m 条边
int cnt = 0;
for (int i = 0; i < m; i++) {
    int a, b, d, p;
    if (scanf("%d%d%d%d", &a, &b, &d, &p) != 4) {
        printf("第 %d 条边读取失败, 请检查 .txt。\\n", i + 1);
        return 0;
    }

    printf("第 %d 条边: (%d, %d) 长度=%d 花费=%d\\n", ++cnt, a, b, d, p);

    // 无向图: 对称赋值

    if (d < D[a][b] || (d == D[a][b] && p < C[a][b])) {
        D[a][b] = D[b][a] = d;
        C[a][b] = C[b][a] = p;
    }
}

```

```

// 读取 s, t
if (scanf("%d%d", &s, &t) != 2) {
    printf("读取起点终点 s,t 失败, 请检查 .txt 最后一行格式.\n");
    return 0;
}
printf("起点: %d  终点: %d\n", s, t);

// 求解
Dijkstra();

return 0;
}

```

5.2 主要源代码片段解释

1) 图的存储: $D[][]$ 与 $C[][]$ 邻接矩阵

- $D[i][j]$ 表示 i 到 j 的距离, INF 表示无边;
- $C[i][j]$ 表示 i 到 j 的花费;
- 无向图对称赋值: $D[a][b]=D[b][a]$, $C[a][b]=C[b][a]$ 。

2) 处理重边: 保留更优边

```

if (d < D[a][b] || (d == D[a][b] && p < C[a][b])) {
    D[a][b] = D[b][a] = d;
    C[a][b] = C[b][a] = p;
}

```

保证输入里即使出现多条 $a-b$ 边, 也不会被“更差的数据”覆盖。

3) 松弛规则: 距离更短直接更新; 距离相同则比较花费

```

if (d < dist[v]) { dist[v] = d; cost[v] = c; }
else if (d == dist[v] && c < cost[v]) { cost[v] = c; }

```

6.实验测试数据和实验结果分析

6.1 测试数据

3 2

1 2 5 6

2 3 4 5

1 3

6.2 程序运行结果

```
D:\Users\86187\Desktop\8207 × + v
读取数据说明
数据格式：
第一行：n m (点编号 1~n)
接着 m 行：a b d p (无向边 a-b, 长度 d, 花费 p)
最后一行：s t (起点 s, 终点 t)

读取到：n = 3, m = 2
第 1 条边：(1, 2) 长度=5 花费=6
第 2 条边：(2, 3) 长度=4 花费=5
起点：1 终点：3

求解说明
规则：优先最短距离；若最短距离有多条路径，则取花费最少的一条
起点：1 终点：3

求解结果
最短距离：9
对应花费：11

-----
Process exited after 0.5954 seconds with return value 0
请按任意键继续. . .
```

7.实验总结与体会

实验收获

通过本实验，我对 Dijkstra 算法的适用条件与核心思想有了更清晰的认识：当图的边权均为非负时，Dijkstra 可以通过“每次选择当前未确定点中距离最小者加入集合”的贪心策略，逐步确定从起点到各顶点的最短距离。相比只记住算法步骤，我更理解了它为什么正确：由于边权非负，已确定集合中的最短距离不会再被后续路径变小，因此可以放心“锁定”这些点。

本题在传统“最短距离”目标之上，又增加了 花费最小这一约束，属于典型的“主目标 + 次目标”的优化问题。我在实现过程中体会到：并不需要重新设计一套复杂算法，只要把“花费”作为与距离同步维护的状态，并在松弛时加入规则——距离更短则更新；距离相同则比较花费并更新更小者——就能在不改变 Dijkstra

主框架的前提下，得到满足题意的结果。这种做法本质上是把最短路问题从单维度扩展为“按字典序比较”的二维最优（先 `dist`，再 `cost`），思路非常通用。此外，本实验也让我意识到：最短路算法的工程实现不仅是“写出 Dijkstra”，还包括输入处理、数据结构选择、边界情况判断等细节。例如图可能不连通、终点不可达、存在重边等情况，都需要在程序中提前考虑并做防护。把这些细节补齐后，程序的鲁棒性明显提升，也更符合实际需求的需求。

遇到的问题及解决办法

存在重边时可能覆盖更优数据

读取图数据时，如果同一对顶点之间存在多条边，简单的直接赋值会导致后读入的边覆盖先读入的边，从而可能把更优（更短/更省）的边替换成更差的边，最终影响最短路结果。对此我在读入阶段加入了筛选规则：始终保留距离更短的边；若距离相同，则保留花费更小的边。这样可以保证邻接矩阵中存储的是两点之间的最优直连边，为后续 Dijkstra 提供正确基础。

邻接矩阵在规模增大时带来空间与时间压力

本实验采用邻接矩阵实现，优点是代码直观、取边权方便，但当 n 较大时空间为 $O(n^2)$ ，同时传统 Dijkstra 选最小 `dist` 的过程也是 $O(n^2)$ ，在更大规模数据下会明显变慢。针对这一点，我的改进思路是：若 n 、 m 更大，应改用**邻接表 + 优先队列（堆）**实现 Dijkstra，将复杂度优化到 $O((n+m)\log n)$ ，并显著降低空间占用。考虑到本实验规模（如 $n \leq 1000$ ）仍在可接受范围内，矩阵方案能够满足实验需求，但有进一步优化的方向。

实验四：查找与排序算法设计及应用

（一）查找算法设计及其应用

1.实验项目名称

二叉排序树有关算法的实现

2.实验目的

本实验旨在掌握二叉排序树（BST）的基本性质与插入建树方法，理解查找过程对应的从根出发的连续分支路径，并实现算法判断给定序列是否为该 BST 中一条合法的查找序列，通过测试验证结果。

3.实验任务描述

编写一个程序，构造一棵二叉排序树 bt，判断一个序列 a 是否为二叉排序树 bt 中一个合法的查找序列（从根结点出发的一条分支路径上的连续结点序列）。

4.任务分析及算法设计思路

4.1 任务分析

题目要求：先构造一棵二叉排序树 bt，再判断序列 a 是否为该 BST 中一个合法的查找序列。这里“合法查找序列”已经明确为：

从根结点出发的一条分支路径上的连续结点序列

因此，序列合法必须同时满足：

- 必须从根结点开始：a[0] 必须等于根结点关键字；
- 必须连续：对每个 a[i]，下一项 a[i+1] 必须是当前结点的左孩子或右孩子（不能跳层、不能跨子树、不能中途换分支）；
- 结点必须存在于树中：若下一项在当前结点的左右孩子中都找不到，立即判定不合法。

这类判定问题与“找某个目标值的完整查找路径”很像，但题目给的是一个路径序列，所以最直接的方法是：沿树指针按序列逐项验证相邻关系，每一步只检查一次是否为当前结点的孩子，不需要重新进行 BST 的值比较查找过程。

4.2 算法基本思路

整体分两步：

(1) 用插入法构造 BST；(2) 验证序列是否为根到某结点的连续路径。

(1) 构造二叉排序树（插入建树）

- 从空树开始，依次插入 `data[]` 中的元素；
- 插入时从根开始比较：小于走左子树，大于走右子树，直到找到空位置插入；
- 若遇到重复关键字(题目未要求处理重复)，本实现选择**忽略重复**，以保证 BST 严格有序。

`InsertBST(root, x):`

```
if root == NULL: root = NewNode(x); return
cur = root
while true:
    if x < cur.key:
        if cur.lchild == NULL: cur.lchild = NewNode(x); return
        cur = cur.lchild
    else if x > cur.key:
        if cur.rchild == NULL: cur.rchild = NewNode(x); return
        cur = cur.rchild
    else:
        return // 重复则不插入
```

(2) 判定序列是否合法

- 若树空或序列长度 ≤ 0 ，直接不合法；
- 若 `a[0] != root.key`，直接不合法；
- 从根开始，用指针 `cur` 指向当前结点：

对每个后续元素 `a[i]`：

若 `cur->lchild` 存在且其 `key` 等于 `a[i]`，则 `cur = cur->lchild`；否则若 `cur->rchild` 存在且其 `key` 等于 `a[i]`，则 `cur = cur->rchild`；否则说明下一项不是当前结点的孩子（跳层/换分支/不存在），不合法。

`IsValidSearchSequence(root, a[0..m-1]):`

```
if root == NULL or m <= 0: return false
if a[0] != root.key: return false

cur = root
for i = 1..m-1:
    if cur.lchild != NULL and cur.lchild.key == a[i]:
        cur = cur.lchild
```

```

        else if cur.rchild != NULL and cur.rchild.key == a[i]:
            cur = cur.rchild
        else:
            return false
    return true

```

复杂度分析

- 建树：平均 $O(n \log n)$ （最坏退化为 $O(n^2)$ ，取决于插入序列是否有序）。
- 序列判定：只沿序列走一遍，时间 $O(m)$ ，额外空间 $O(1)$ 。

5.程序结构及代码解析

5.1 程序结构

BSTNode: BST 结点结构体，包含 key、lchild、rchild;

NewNode(int x): 申请并初始化新结点;

InsertBST(BSTNode* &root, int x): 向 BST 插入关键字 x (迭代实现，重复忽略);

IsValidSearchSequence(BSTNode* root, int a[], int m): 判定序列 a 是否为从根出发的连续分支路径;

DestroyBST(BSTNode* &root): 后序递归释放整棵树;

PrintSeq(int a[], int m): 输出数组，便于展示测试序列;

main(): 建树→输出建树数据→对多组测试序列进行合法性判断→释放内存。

```

#include <stdio.h>
#include <stdlib.h>

typedef struct BSTNode {
    int key;
    struct BSTNode *lchild;
    struct BSTNode *rchild;
} BSTNode;

BSTNode* NewNode(int x) {
    BSTNode* p = (BSTNode*)malloc(sizeof(BSTNode));
    p->key = x;

```

```

    p->lchild = p->rchild = NULL;
    return p;
}

void InsertBST(BSTNode* &root, int x) {
    if (root == NULL) {
        root = NewNode(x);
        return;
    }
    BSTNode* cur = root;
    while (1) {
        if (x < cur->key) {
            if (cur->lchild == NULL) {
                cur->lchild = NewNode(x);
                return;
            }
            cur = cur->lchild;
        } else if (x > cur->key) {
            if (cur->rchild == NULL) {
                cur->rchild = NewNode(x);
                return;
            }
            cur = cur->rchild;
        } else {
            return; // 忽略重复
        }
    }
}

int IsValidSearchSequence(BSTNode* root, int a[], int m) {
    if (root == NULL || m <= 0) return 0;
    if (a[0] != root->key) return 0;

    BSTNode* cur = root;
    for (int i = 1; i < m; i++) {
        if (cur->lchild && cur->lchild->key == a[i])
            cur = cur->lchild;
        else if (cur->rchild && cur->rchild->key == a[i])
            cur = cur->rchild;
        else
            return 0;
    }
}

```

```

        return 1;
    }

void DestroyBST(BSTNode* &root) {
    if (root) {
        DestroyBST(root->lchild);
        DestroyBST(root->rchild);
        free(root);
        root = NULL;
    }
}

void PrintSeq(int a[], int m) {
    for (int i = 0; i < m; i++) printf("%d ", a[i]);
}

int main() {
    BSTNode* bt = NULL;

    /* ===== 4 层左右的 BST ===== */
    int data[] = {50, 30, 70, 20, 40, 60, 80, 10, 25, 35, 45};
    int n = sizeof(data) / sizeof(data[0]);

    for (int i = 0; i < n; i++)
        InsertBST(bt, data[i]);

    printf("已构造二叉排序树 bt (共 %d 个结点, 约 4 层): \n", n);
    PrintSeq(data, n);
    printf("\n\n");

    int t1[] = {50, 30, 20, 10};
    int t2[] = {50, 30, 40, 45};
    int t3[] = {50, 70, 60};
    int t4[] = {50, 70, 80};

    int t5[] = {50, 20};          // 跳层
    int t6[] = {30, 20};          // 非根开始
    int t7[] = {50, 30, 70};      // 中途换分支
    int t8[] = {50, 30, 99};      // 不存在
    int t9[] = {50, 40, 35};      // 孙子当孩子

    int* tests[] = {t1, t2, t3, t4, t5, t6, t7, t8, t9};
    int lens[] = {4, 4, 3, 3, 2, 2, 3, 3, 3};

```

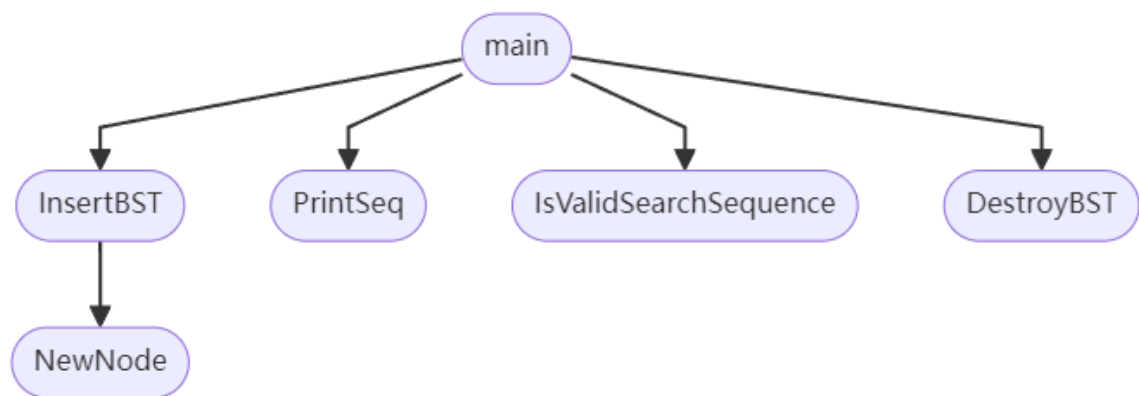
```

printf("\n 查找序列合法性判定\n");
for (int i = 0; i < 9; i++) {
    printf("序列 %d: ", i + 1);
    PrintSeq(tests[i], lens[i]);
    printf("\n 结果: %s\n\n",
        IsValidSearchSequence(bt, tests[i], lens[i]) ? "YES" : "NO");
}

DestroyBST(bt);
return 0;
}

```

5.2 函数调用关系图

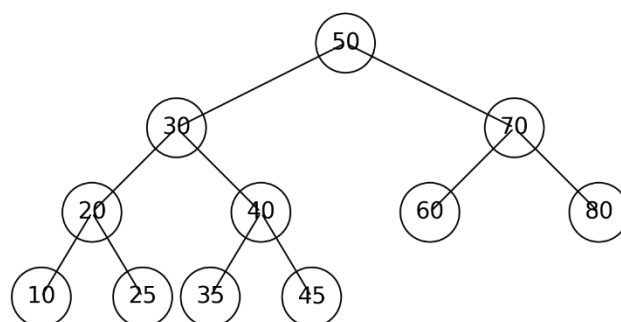


6. 实验测试数据和实验结果分析

6.1 测试数据

(1) 构造 BST 的插入序列 (约 4 层):

data = {50, 30, 70, 20, 40, 60, 80, 10, 25, 35, 45}



(2) 待判定的查找序列:

合法示例 (沿单一分支连续向下):

- $t1 = \{50, 30, 20, 10\}$
- $t2 = \{50, 30, 40, 45\}$
- $t3 = \{50, 70, 60\}$
- $t4 = \{50, 70, 80\}$

不合法示例:

- 跳层: $t5 = \{50, 20\}$
- 非根开始: $t6 = \{30, 20\}$
- 中途换分支: $t7 = \{50, 30, 70\}$
- 结点不存在: $t8 = \{50, 30, 99\}$
- 孙子当孩子: $t9 = \{50, 40, 35\}$

6.2 程序运行结果

```
D:\Users\86187\Desktop\8207 × + v
已构造二叉排序树 bt (共 11 个结点, 约 4 层):
50 30 70 20 40 60 80 10 25 35 45

查找序列合法性判定
序列 1: 50 30 20 10
结果: YES

序列 2: 50 30 40 45
结果: YES

序列 3: 50 70 60
结果: YES

序列 4: 50 70 80
结果: YES

序列 5: 50 20
结果: NO

序列 6: 30 20
结果: NO

序列 7: 50 30 70
结果: NO

序列 8: 50 30 99
结果: NO

序列 9: 50 40 35
结果: NO

-----
Process exited after 0.4342 seconds with return value 0
请按任意键继续. . . |
```

7.实验总结与体会

实验收获

通过本实验，我对二叉排序树的结构特征和查找过程的“路径性”有了更直观的理解：BST 的一次查找必然从根开始，根据比较结果在左/右子树中不断向下推进，因此访问到的结点一定构成一条连续的分支路径。题目把“查找序列”抽象成路径上的连续结点序列，使我意识到：有时不必“重新做一遍查找”，而可以直接利用树的指针结构对序列进行逐项验证，从而将判定过程简化为线性扫描。

同时，我也体会到测试用例设计的重要性。为了覆盖常见错误情形，我刻意加入了“跳层、换分支、非根开始、不存在结点、孙子当孩子”等反例，这些反例能有效检验算法是否真正符合题意，而不是仅仅对少量正例有效。通过对比输出，我更清楚地知道了“连续”这一要求在程序里具体该如何体现。

遇到的问题及解决办法

跳层与换分支的判定容易遗漏：若仅用能不能在树里找到这些值来判断，会把 {50, 20} 或 {50, 30, 70} 误判为合法。必须在循环中严格限制每一步只能走到 cur 的左右孩子，不能从任意位置继续搜索。

内存管理问题：动态申请结点如果不释放会造成内存泄漏。
增加 DestroyBST() 后序释放函数，在程序末尾释放整棵树，保持程序规范与可复用性。

对合法查找序列的含义容易理解偏差，一开始容易把它理解成“只要序列中的值满足 BST 的大小关系即可”，但题目强调的是“从根出发的一条分支路径上的连续结点序列”。把要求落到结构层面，用指针检查下一项是否为当前结点的左/右孩子，而不是仅用大小比较去推断。

（二）排序算法设计及其应用

1. 实验项目名称

不同排序算法的测试与比较

2. 实验目的

通过随机生成整数序列，分别采用冒泡排序、直接插入排序与快速排序对同一数据进行递增排序，通过对比三种算法的排序过程，理解不同排序算法在时间复杂度、数据分布敏感性以及实际运行效率上的差异，为后续在不同场景下选择合适排序方法提供依据。

3. 实验任务描述

编写一个程序，随机产生 n 个 $1 \sim 999$ 的正整数序列，分别采用 3 种以上不同的排序算法对其进行递增排序，输出各趟的排序结果，并求出每种排序算法所需要的绝对时间。

4. 任务分析及算法设计思路

4.1 任务分析

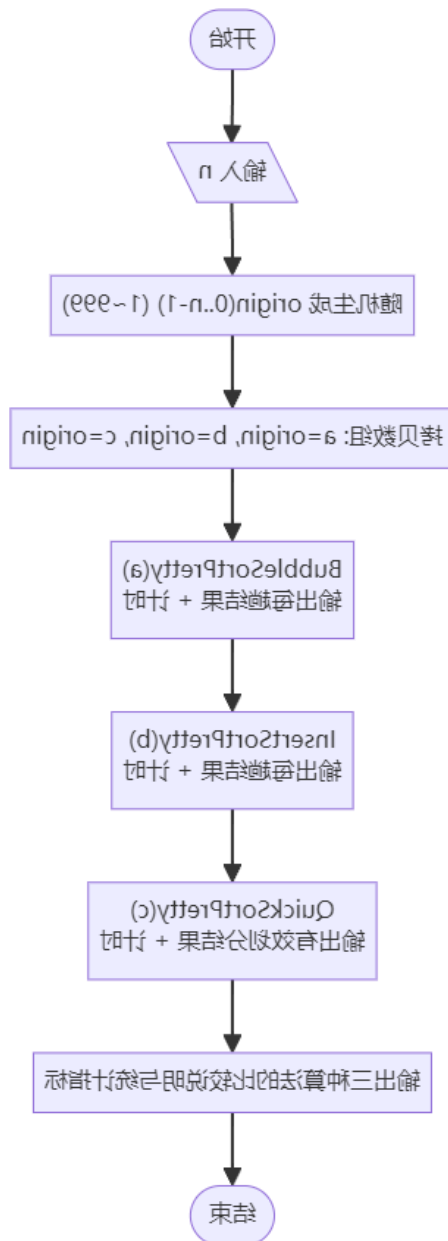
实验任务包含三个核心点：

- 1) 数据生成：随机产生 n 个 $1 \sim 999$ 的正整数，作为原始序列 `origin[]`；
- 2) 多算法排序对比：至少 3 种排序算法对同一序列排序。为了保证公平比较，需要先把 `origin[]` 拷贝成多个数组（如 `a/b/c`），分别执行不同算法；
- 3) 过程输出与时间统计：

输出各趟结果要求排序过程中按趟打印数组状态：冒泡/插入按趟输出，快排按划分输出（本程序只输出数组确实发生变化的划分，避免冗余）；

绝对时间用 `clock()` 记录起止时间并换算为毫秒（ms）。

4.2 总体流程



4.3 三种排序算法思路概述

(1) 冒泡排序

每一趟把当前未排序部分的最大值“冒”到末尾；若某一趟无交换，说明已整体有序，可提前结束。

```

for i = 0..n-2:
    flag = false
    for j = 0..n-2-i:
        if a[j] > a[j+1]:
            swap(a[j], a[j+1]); flag=true
    输出第 i+1 趟数组
    if flag == false: break
  
```

(2) 直接插入排序

把 $a[i]$ 作为 key，向前扫描有序区，把比 key 大的元素后移，为 key 找到插入位置。

```
for i = 1..n-1:
    key = a[i]
    j = i-1
    while j>=0 and a[j] > key:
        a[j+1] = a[j]    // 后移
        j--
    a[j+1] = key
    输出第 i 趟数组
```

(3) 快速排序

选定枢轴 pivot，把序列划分为 “ $\leq \text{pivot}$ ” 和 “ $\geq \text{pivot}$ ” 两部分，然后递归处理左右子区间。

```
QuickSort(low, high):
    if low >= high: return
    pivotPos = Partition(low, high)
    输出本次有效划分后的数组（若数组有变化）
    QuickSort(low, pivotPos-1)
    QuickSort(pivotPos+1, high)
```

5. 程序结构及代码解析

5.1 程序结构

PrintArray(a, n): 按列打印数组，便于观察每趟变化；

CopyArray(dst, src, n): 将原始序列拷贝到各算法数组，确保对比公平；

IsSorted(a, n): 判断是否有序（插入排序中可选用作“整体提前结束”）；

BubbleSortPretty(a, n): 冒泡排序，每趟输出，统计比较/交换次数与耗时；

InsertSortPretty(a, n): 直接插入排序，每趟输出，统计比较/移动次数与耗时；

QuickSortPretty(a, n): 快排入口，计时并调用递归核心；

QuickSortCorePretty(a, low, high, ...): 快排递归核心；

Partition(a, low, high, ...): 划分函数，完成一次分区并返回枢轴位置；

ArraysEqual(before, a, n): 用于判断一次划分是否使数组发生变化(避免输出冗余)。

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
```

```

#define MAXN 200

void PrintArray(int a[], int n) {
    for (int i = 0; i < n; i++) printf("%4d", a[i]);
    printf("\n");
}

int IsSorted(int a[], int n) {
    for (int i = 1; i < n; i++) if (a[i-1] > a[i]) return 0;
    return 1;
}

void CopyArray(int dst[], int src[], int n) {
    for (int i = 0; i < n; i++) dst[i] = src[i];
}

//冒泡排序: 无交换提前结束
void BubbleSortPretty(int a[], int n) {
    int i, j, temp;
    int pass = 0;
    long long cmp = 0, swp = 0;
    clock_t start, end;

    printf("\n 冒泡排序:每趟结果 (若某趟无交换则提前结束): \n");
    printf("初始: ");
    PrintArray(a, n);

    start = clock();

    for (i = 0; i < n - 1; i++) {
        int flag = 0;
        for (j = 0; j < n - 1 - i; j++) {
            cmp++;
            if (a[j] > a[j + 1]) {
                temp = a[j]; a[j] = a[j + 1]; a[j + 1] = temp;
                swp++;
                flag = 1;
            }
        }
        pass++;
        printf("第 %d 趟: ", pass);
    }
}

```

```

        PrintArray(a, n);

        if (flag == 0) { // 本趟没有发生交换, 说明已经有序
            printf(" (本趟无交换, 排序已完成, 提前结束) \n");
            break;
        }
    }

    end = clock();

    printf("结果: ");
    PrintArray(a, n);
    printf("趟数=%d, 比较次数=%lld, 交换次数=%lld, 用时=%.3f ms\n",
        pass, cmp, swp, 1000.0 * (end - start) / CLOCKS_PER_SEC);
}

//插入排序: 每趟输出, 统计移动次数
void InsertSortPretty(int a[], int n) {
    int i, j, key;
    int pass = 0;
    long long cmp = 0, mov = 0;
    clock_t start, end;

    printf("\n 直接插入排序: 每趟结果: \n");
    printf("初始: ");
    PrintArray(a, n);

    start = clock();

    for (i = 1; i < n; i++) {
        key = a[i];
        j = i - 1;
        int movedThisPass = 0;

        while (j >= 0) {
            cmp++;
            if (a[j] > key) {
                a[j + 1] = a[j];
                mov++;
                movedThisPass = 1;
                j--;
            } else {
                break;
            }
        }
    }
}

```

```

    }
    a[j + 1] = key;

    pass++;
    printf("第 %d 趟: ", pass);
    PrintArray(a, n);
    if (!movedThisPass) {
        printf(" (本趟无需移动元素) \n");
    }

    // 如果你也想“完全有序就提前停”，打开下面两行：
    // if (IsSorted(a, n)) { printf(" (整体已排序, 提前结束) \n"); break; }
}

end = clock();

printf("结果: ");
PrintArray(a, n);
printf("趟数=%d, 比较次数=%lld, 移动次数=%lld, 用时=%.3f ms\n",
        pass, cmp, mov, 1000.0 * (end - start) / CLOCKS_PER_SEC);
}

//快速排序: 只在数组变化时输出
int ArraysEqual(int a[], int b[], int n) {
    for (int i = 0; i < n; i++) if (a[i] != b[i]) return 0;
    return 1;
}

int Partition(int a[], int low, int high, long long *cmp, long long *mov) {
    int pivot = a[low];
    while (low < high) {
        while (low < high) { (*cmp)++; if (a[high] >= pivot) high--; else break; }
        if (low < high) { a[low] = a[high]; (*mov)++; low++; }

        while (low < high) { (*cmp)++; if (a[low] <= pivot) low++; else break; }
        if (low < high) { a[high] = a[low]; (*mov)++; high--; }
    }
    a[low] = pivot; (*mov)++;
    return low;
}

void QuickSortCorePretty(int a[], int low, int high, int n,
                          int *cnt, long long *cmp, long long *mov) {
    if (low >= high) return;

```

```

    int before[MAXN];
    for (int i = 0; i < n; i++) before[i] = a[i];

    int pivot = Partition(a, low, high, cmp, mov);

    if (!ArraysEqual(before, a, n)) {
        (*cnt)++;
        printf("第 %d 次划分: ", *cnt);
        PrintArray(a, n);
    }

    QuickSortCorePretty(a, low, pivot - 1, n, cnt, cmp, mov);
    QuickSortCorePretty(a, pivot + 1, high, n, cnt, cmp, mov);
}

void QuickSortPretty(int a[], int n) {
    clock_t start, end;
    int cnt = 0;
    long long cmp = 0, mov = 0;

    printf("\n 快速排序:每次有效划分结果: \n");
    printf("初始: ");
    PrintArray(a, n);

    start = clock();
    QuickSortCorePretty(a, 0, n - 1, n, &cnt, &cmp, &mov);
    end = clock();

    printf("结果: ");
    PrintArray(a, n);
    printf("有效划分次数=%d, 比较次数=%lld, 移动次数=%lld, 用时=%.3f ms\n",
        cnt, cmp, mov, 1000.0 * (end - start) / CLOCKS_PER_SEC);
}

int main() {
    int n;
    int origin[MAXN], a[MAXN], b[MAXN], c[MAXN];

    printf("请输入 n (建议 <= 30, 便于观察输出): ");
    if (scanf("%d", &n) != 1 || n <= 0 || n > MAXN) {
        printf("n 输入不合法. \n");
    }
}

```

```

        return 0;
    }

    srand((unsigned)time(NULL));

    printf("\n 随机生成的序列 (1~999): \n");
    for (int i = 0; i < n; i++) {
        origin[i] = rand() % 999 + 1;
        printf("%4d", origin[i]);
    }
    printf("\n");

    CopyArray(a, origin, n);
    CopyArray(b, origin, n);
    CopyArray(c, origin, n);

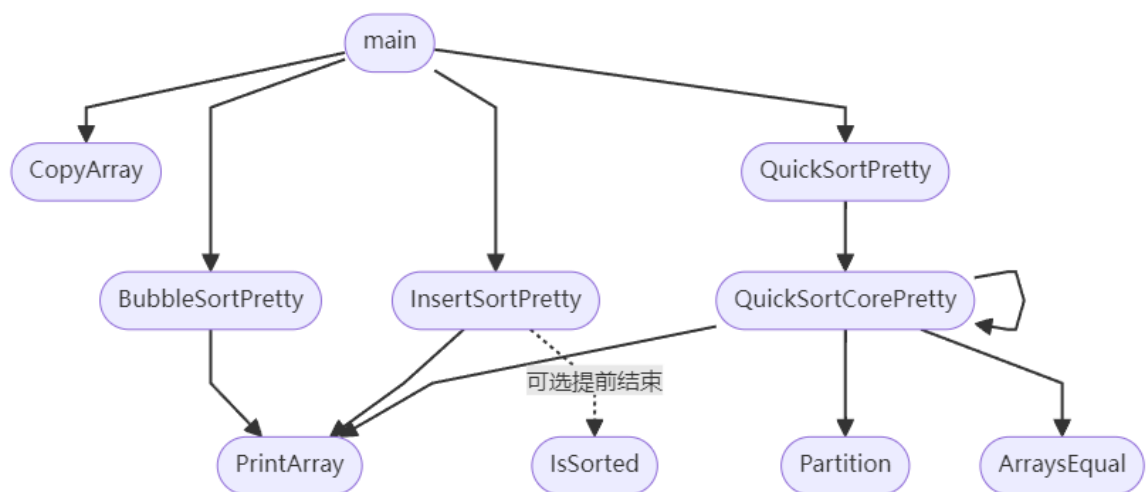
    BubbleSortPretty(a, n);
    InsertSortPretty(b, n);
    QuickSortPretty(c, n);

    printf("\n 比较说明:\n");
    printf("1) 冒泡: 若某趟无交换则提前结束; 一般交换多、效率低。 \n");
    printf("2) 插入: 趟数通常为 n-1, 但当序列接近有序时移动次数很少。 \n");
    printf("3) 快排: 递归分区, 平均效率高; 本程序只输出有效划分避免冗余。 \n");

    return 0;
}

```

5.2 函数调用关系图



6.实验测试数据和实验结果分析

6.1 测试一：n = 36

(1) 测试数据

输入 n=36，随机生成序列为：

284 27 906 315 21 899 513 543 980 329 370 993 667 657 737 694 386
351 736 530 328 925

(2) 运行结果与统计指标

三种算法最终排序结果一致，得到递增序列：

21 27 190 209 213 247 279 284 315 669 694 736 737 761 803 895 899
906 913 925 928 980 986 993

- **冒泡排序：**趟数=26（第 26 趟无交换提前结束），比较次数=585，交换次数=270，用时=31.000 ms
- **直接插入排序：**趟数=35，比较次数=303，移动次数=270，用时=42.000 ms
- **快速排序：**有效划分次数=18，比较次数=161，移动次数=84，用时=16.000 ms

(3) 结果分析

从统计指标看，快速排序的比较次数与移动次数明显更少，因此耗时最低；冒泡排序虽然加入“无交换提前结束”，但仍有大量相邻比较与交换；插入排序在本次数据中出现多次“无需移动元素”的趟次，说明其对局部有序有优势，但总体移动次数仍较多，时间略高于冒泡。总体结论是：在本次 n=36 的随机数据下，快排综合效率最好。

输入 n=100，随机生成序列见运行截图

(2) 运行结果与统计指标

- 冒泡排序：趟数=95，比较次数=4940，交换次数=2340，用时=240.000 ms
- 直接插入排序：趟数=99，比较次数=2435，移动次数=2340，用时=391.000 ms
- 快速排序：有效划分次数=48，比较次数=616，移动次数=283，用时=193.000 ms

(3) 结果分析

当 n 增大到 100 后，三种算法的差异更加明显：

- 冒泡排序比较次数接近 $n(n-1)/2$ 的量级（这里为 4940），体现其典型的平方级开销；虽然有提前结束，但对随机数据帮助有限。
- 插入排序比较次数明显少于冒泡（2435），但移动次数仍达到 2340，且趟数固定为 $n-1=99$ ；在本次运行环境下，耗时反而高于冒泡，说明“比较次数少”不一定意味着“总时间更少”，还与移动操作、循环结构、缓存与实现细节有关。
- 快速排序的比较与移动数量最低（616/283），体现分治算法在随机数据下的优势，因此用时最短（193 ms）。

```
D:\Users\86187\Desktop\8207 > + >
请输入 n (建议 <= 30, 便于观察输出) : 100

随机生成的序列 (1~999) :
542 180 155 773 668 215 38 309 294 293 582 397 466 645 190 375 359 575 663 677 552 132 510 423 132 890
886 507 752 487 93 984 163 723 772 594 547 660 915 637 378 139 208 440 336 143 438 226 193 230 293 816
6 467 81 195 329 74 673 746 591 217 628 214 928 981 589 179 644 125 910 108 572 259 10 571 703 476 99
5 283 989 849 276 963 447 216 84 459 281 494 243 956 972 344 695 891 31 585 518 395

冒泡排序:每趟结果 (若某趟无交换则提前结束) :
初始: 542 180 155 773 668 215 38 309 294 293 582 397 466 645 190 375 359 575 663 677 552 132 510 423 13
2 890 886 507 752 487 93 984 163 723 772 594 547 660 915 637 378 139 208 440 336 143 438 226 193 230 293

第 95 趟: 6 10 31 38 74 81 84 93 108 125 132 132 139 143 155 163 179 180 190 193 195 208 214 21
5 216 217 226 230 243 259 276 281 283 293 293 294 309 329 336 344 359 375 378 395 397 423 438 440 447 459
466 467 476 487 494 507 510 518 542 547 552 571 572 575 582 585 589 591 594 628 637 644 645 660 663 668
673 677 695 703 723 746 752 772 773 816 849 886 890 891 910 915 928 956 963 972 981 984 989 995
(本趟无交换, 排序已完成, 提前结束)
结果: 6 10 31 38 74 81 84 93 108 125 132 132 139 143 155 163 179 180 190 193 195 208 214 215 21
6 217 226 230 243 259 276 281 283 293 293 294 309 329 336 344 359 375 378 395 397 423 438 440 447 459 466
467 476 487 494 507 510 518 542 547 552 571 572 575 582 585 589 591 594 628 637 644 645 660 663 668 673
677 695 703 723 746 752 772 773 816 849 886 890 891 910 915 928 956 963 972 981 984 989 995
趟数=95, 比较次数=4940, 交换次数=2340, 用时=240.000 ms

5 216 217 226 230 243 259 276 281 283 293 293 294 309 329 336 344 359 375 378 395 397 423 438 440 447 459
466 467 476 487 494 507 510 518 542 547 552 571 572 575 582 585 589 591 594 628 637 644 645 660 663 668
673 677 695 703 723 746 752 772 773 816 849 886 890 891 910 915 928 956 963 972 981 984 989 995
结果: 6 10 31 38 74 81 84 93 108 125 132 132 139 143 155 163 179 180 190 193 195 208 214 215 21
6 217 226 230 243 259 276 281 283 293 293 294 309 329 336 344 359 375 378 395 397 423 438 440 447 459 466
467 476 487 494 507 510 518 542 547 552 571 572 575 582 585 589 591 594 628 637 644 645 660 663 668 673
677 695 703 723 746 752 772 773 816 849 886 890 891 910 915 928 956 963 972 981 984 989 995
趟数=99, 比较次数=2435, 移动次数=2340, 用时=391.000 ms
```

```

4 215 216 217 226 230 243 259 276 281 283 293 293 294 309 329 336 344 359 375 378 395 397 423 438 440 447
459 466 467 476 487 494 507 510 518 542 547 552 571 572 575 582 585 589 591 594 628 637 644 645 660 663
668 673 677 695 703 723 746 752 772 773 816 849 886 890 891 910 915 928 956 963 972 981 984 989 995
结果: 6 10 31 38 74 81 84 93 108 125 132 132 139 143 155 163 179 180 190 193 195 208 214 215 21
6 217 226 230 243 259 276 281 283 293 293 294 309 329 336 344 359 375 378 395 397 423 438 440 447 459 466
467 476 487 494 507 510 518 542 547 552 571 572 575 582 585 589 591 594 628 637 644 645 660 663 668 673
677 695 703 723 746 752 772 773 816 849 886 890 891 910 915 928 956 963 972 981 984 989 995
有效划分次数=48, 比较次数=616, 移动次数=283, 用时=193.000 ms
比较说明:
1) 冒泡: 若某趟无交换则提前结束; 一般交换多、效率低。
2) 插入: 趟数通常为  $n-1$ , 但当序列接近有序时移动次数很少。
3) 快排: 递归分区, 平均效率高; 本程序只输出有效划分避免冗余。

```

7. 实验总结与体会

实验收获

本实验以同一随机序列为输入，分别采用冒泡排序、直接插入排序和快速排序完成递增排序，并输出关键中间过程与统计指标（趟数/比较次数/交换或移动次数/耗时）。在看得见过程的前提下，我对三类排序算法的工作机制有了更直观的理解。

通过对 $n=36$ 与 $n=100$ 两组数据的对比，我更清晰地感受到了算法的“规模效应”。当 n 较小时，三种算法都能较快完成排序，但从比较/移动（交换）次数已经能看出差异；当 n 增大到 100 后，差异更加明显：冒泡排序需要大量相邻比较与交换，整体开销随规模增长较快；插入排序在序列接近有序时会显著减少移动，表现较好，但在随机数据下移动操作仍较多，效率会波动；快速排序在两组测试中比较次数与移动次数都更少，表现稳定且整体效率最好，体现了其平均情况下的优势。尤其在 $n=100$ 时，快排的优势更突出，说明随着数据规模增加，算法的渐近复杂度会逐渐成为影响实际运行时间的主导因素。

此外，本实验也让我认识到：评价排序算法不能只看最终是否有序，更要关注过程中的基本操作次数以及输出与计时的实验设计方式。比较次数、移动/交换次数通常比绝对时间更稳定；而时间数据则会受到编译优化、系统负载、输出开销等因素影响，因此在实验报告中应结合两类指标给出更可靠的结论。

遇到的问题及解决办法

当 n 较大时，冒泡和插入按“每趟输出”会导致终端刷屏，阅读困难，并且 I/O 输出本身会显著增加耗时，使计时结果混入打印开销。解决方法是：将实验分为两种模式——小规模（如 $n \leq 30$ ）用于观察排序过程与中间结果；大规模（如 $n=100/500/1000$ ）用于性能统计时减少或关闭逐趟输出，仅保留最终结果与统计信息，必要时只输出关键阶段（例如每 10 趟输出一次）。

当 n 很小时，排序耗时可能低于 `clock()` 的分辨率，出现 0.000 ms 或波动较大的情况；当 n 较大时，又可能受到系统负载、编译优化选项等影响，导致同一程序多次运行时间不同。解决方法是：适当增大 n ，并采用“重复多次取平均”的方式（例如循环排序 100 次求平均耗时），同时在报告中优先引用比较次数、移动/交换次数作为更稳定的对比依据，从而让结论更可信。

快排递归层次多，如果每次递归都输出数组，日志会非常长且信息重复。解决方法是：本程序采用“仅输出数组发生变化的有效划分”的策略，即在一次 `partition` 前后比较数组是否变化，只有变化时才打印，这样既能保留快排关键过程（分区效果），又能避免大量无意义输出，提高可读性。